F&P Personal Robotics

myP User Manual

Version 1.4.4

(corresponds to the myP version)

2020-03-16

The information contained in this document is property of F&P Robotics AG and shall not be reproduced in whole or in part without prior written approval of F&P Robotics AG. The information provided herein is subject to change without notice and should not be constructed as a commitment by F&P Robotics AG. This manual is periodically reviewed and revised. F&P Robotics AG assumes no responsibility for any errors or omissions in this document.

This manual consists of original instructions from F&P Robotics AG and is intended for all users of myP. Before using myP with a P-Rob robotic arm or P-Grip robotic gripper, you must read the corresponding user manuals carefully, paying particular attention to the safety instructions in the P-Rob user manual. Compliance with the instructions in all manuals is mandatory.

Copyright © 2011-2020 by F&P Robotics AG. All rights reserved.

The F&P logo as well as P-Rob®, P-Grip® and myP® are registered trademarks of F&P Robotics AG.

Contents

Introduction **5**

Programming & Controlling 5

Graphical User Interface (GUI) 5

Robotic Operating System (ROS) 6

Software Add-Ons 6

Learning Module 7

Cognex Module 7

TCP Connection 7

Modbus Connection 8

Setup & Updates **9**

Installation 9

Starting up P-Rob 9

Connecting to myP 9

Updating myP and P-Rob 9

Set Update Configuration 10

Update P-Rob to New Version 11

Restore myP Backup 11

Network Setup 12

Set Network Configuration 12

Get Network Configuration 12

Desktop Interface (GUI) **13**

Main Window 14

Cockpit 15

Information & Help 16

Message History 17

Program Status 18
Menu Overview 19
Robot State 19
Application Output 19
Building Blocks & Database 20
Project Menu 21
Connection Menu 22
Connecting P-Rob 22
Calibrating P-Rob 22
Shut-down and Reboot 23
Database Download and Upload 23
Teach-In Menu 24
Pose Editor 26
Path Editor 28
Grid Editor 30
Application Menu 33
Application Editor 34
Application Code 34
Application Settings 35
Application Variables 37
Shared & Global Variables 38
Sensors Menu 39
Settings Menu 40
Language Settings 40
Start-Up Settings 41
Collision Detection Settings 41
Panel Interface (GUI) 42

Configure Applications for Panel Interface 43
Cockpit and Settings 44
Controlling Applications 45
Robot Calibration 46
Default Calibration 46
Custom Calibration 47
Mathematical Conventions 48
Coordinate Systems 48
Pose 49
Orientation Convention 49
Synchronizing Multiple P-Robs 50
TCP Socket 50
Digital I/Os 52

1. Introduction

This document guides the user through the basic usage of myP. It is intended for all users of a P-Rob robotic arm.

Programming & Controlling

The software framework myP provides the user of a personal robotic arm P-Rob and its gripper P-Grip with a very intuitive way to communicate with this robot. Using myP, the user has full access to manipulate and control the robot as well as writing program applications using the programming language Python 3.5.

In addition to a huge variety of basic Python functions, myP applications support a large set of functions specifically designed for controlling, maintaining, and communicating with P-Robs. You can find a complete list of all application programming functions in the myP Script Functions Manual.

Graphical User Interface (GUI)

The GUI of myP has been designed as a web application that is able to run on common web browsers of basically any devices. The following browsers are officially supported:

Therefore, there is no need to download a specific software module for your controlling device, as long as the robot and device are in the same network or share a direct communication channel. In addition to the default desktop interface of myP, which is mainly optimized for programming on desktop computers and laptops, myP provides a Panel Interface, which comes with only the necessary functions for operating an already pre-programmed P-Rob. You will find more information about the GUIs and all of their supported functions further down in this document.

Robotic Operating System (ROS)

In addition to the graphical interface, we also provide the user with an add-on containing functional support for the Robotic Operating System (ROS).

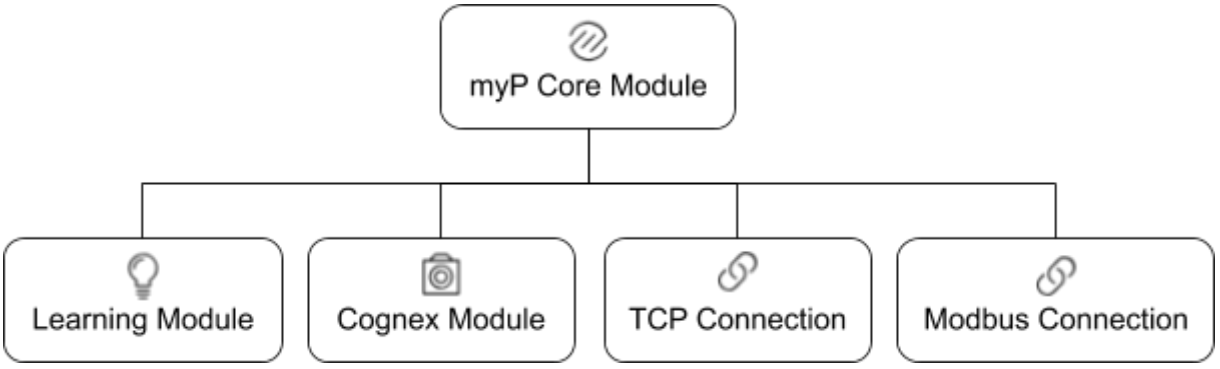
myP publishes data about the robot to ROS topics and provides ROS services for various control functions. In order to use myP via ROS, you will need to install our custom ROS messages, which you will find on the F&P GitHub repositories:

- Core Messages (necessary to control P-Rob via ROS)
- P-Grip Messages (if you want to control a P-Grip via ROS in addition)
- Learning Messages (if you are using the myP Learning Module in addition)

Please note that this document does not cover the communication between ROS and myP. For details about the ROS communication and a full function description, please refer to the myP ROS Module Manual.

Software Add-Ons

In addition to the basic functionality of myP, which is explained in detail throughout this document, F&P Robotics develops many different add-ons for myP to allow for more versatile usage and extended functionality range. Please contact F&P Robotics if you are interested in buying add-ons or if you'd like to get more detailed information about specific add-ons, please contact sales@fp-robotics.com.



Please note that this document does not cover the myP add-ons and their functionality. For details about a specific add-on, including a full list of functions and examples, refer to the add-on specific manuals.

Learning Module

The Learning module combines a set of smart functions into one package. On one side the user is able to teach objects to P-Rob using the integrated sensors of a P-Grip, and P-Rob will distinguish/recognize those objects on run-time. On the other hand this module comes with the Task Generator, which is a smart tool for recording movements and generating pick&place tasks.

Cognex Module

The new Cognex module provides a communication interface between a Cognex camera (which can be integrated into P-Grip) and myP. This module is perfect for customizable visual inspection, supporting the full function range of Cognex cameras.

TCP Connection

The TCP Connection module can be used as an alternative way to communicate with the robot. The robot is usually controlled by the user through the myP Front End in the web browser. However, if the robot is supposed to be controlled by a custom interface or by another machine, it directly accepts tcp commands. With the tcp connection module, everything that can be achieved by the browser, can also be achieved via TCP.

Modbus Connection

The Modbus Connection module can be used as an alternative way to communicate with the robot. It uses the Modbus tcp protocol and lets the user read and write standard modbus registers. Use this addon to directly control the robot via a PLC. The most important functionalities such as connecting to the motors, calibrating the robot and running scripts can be achieved by the modbus module.

2. Setup & Updates

Installation

P-Rob already comes pre-installed with myP, so you don't have to worry about installation.

Starting up P-Rob

Make sure that P-Rob is properly powered and cabled according to the P-Rob User Manual. Then simply press the front button on the base of P-Rob to start it up. During the start-up phase, the button will blink blue. Once the blinking turns into a static blue light, P-Rob is ready to listen to your commands.

Connecting to myP

myP has been designed to run as a web application. Thus, you first need to figure out the IP address of your robot. The IP address may be either fixed or automatically assigned. If you connect to the robot through a local network where IP addresses are assigned automatically via DHCP, you need to refer to your system administrator or read the IP of the P-Rob using an update stick. Make sure that your device is in the same network as P-Rob, and that the network settings of both your device and the robot are matching.

Please note that newer versions of P-Rob come with two Ethernet ports, of which the left one (labelled **Ethernet 2**) is customizable and the right one (labelled **Ethernet 3**) comes with the fixed static maintenance IP address **10.0.0.203** that cannot be changed by the administrator.

Once the network connection is established and the IP address of P-Rob is known, you can continue connecting to myP. You need to have a web browser installed on your computer or controlling device. Check out the previous chapter for a list of officially supported web browsers. Enter the IP address of your P-Rob in the URL of your web browser, which will forward you to **https://<IP>:8889/desktop**, which is the main Desktop Interface of myP. If you'd rather like to access another interface of myP (e.g. the Panel Interface), you may switch to **https://<IP>:8889/panel** instead.

Updating myP and P-Rob

To install software updates, install software packages, or to configure your robot, you will need a USB stick with at least 1 GB of free memory which is formatted with FAT32 or NTFS. Please note that sticks formatted with extFAT won't be detected!

Extract the provided file **myP_update.zip** onto this memory stick. When the extraction is completed, check that you find the file **stick-config** and the folder **update-prob** in the root directory of the USB stick. Now the update stick is ready to use.

Set Update Configuration

Open the **stick-config** file in the root directory of the USB stick. The first half of this file is dedicated to the P-Rob update procedure. Inside you'll find the following content:

```
[UPDATE_PROB]
# Execute update of software and firmware of P-Rob.
# 1: Updates the P-Rob, 0: Does not update the P-Rob
update_prob = 1

[IP_CONFIG]
# Read the IP of P-Rob in your network.
# 1: Prints P-Rob IP to file PROB_IP in the root of the update stick, 0: Does not print the IP
get_ip = 1

# Change P-Rob network configuration.
# On P-Robs of version d and newer, this changes the configuration of the 'Ethernet 2' port.
# On older P-Robs this changes the configuration of the 'Ethernet' port.
# 1: Changes the network configuration according to the settings below, 0: Does not change the
network configuration
change_ip_config = 0

# Network configuration:
# Set the port to a static IP or DHCP
# static: Use static IP, dhcp: use external DHCP server
protocol = static

# Settings for static IP configuration (not applied for protocol = dhcp)
# Note: These are example settings, ask your local network administrator for the correct settings!
ip_address = 192.168.1.203
netmask = 255.255.255.0
gateway = 192.168.1.1
dns = 192.168.1.1


[ADVANCED]
# Force P-rob update even if backup of active installation fails.
# 1: Updates the P-Rob always, 0: Only updates the P-Rob if backup succeeds
update_prob_force = 0

# Reset myP database.
# 1: Deletes existing myP database, 0: Keeps existing database
reset_database = 0

# Delete existing myP config.
# 1: Delete user myP config files, 0: Preserve myP config files
reset_config = 0

# Delete myP data folder
# 1: Deletes existing myP data and installs updated version, 0: Keep existing myP data folder
reset_myP_data = 0

# Restore an old myP installation.
# 1: enable restore (deactivates P-Rob update), 0: restore is disabled.
restore_myP = 0
```

To enable the P-Rob update procedure, set the parameter **update_prob** in the section **[UPDATE_PROB]** to 1 and make sure that **restore_myp** in the **[ADVANCED]** section is set to 0 (default). If you want the configuration files or the database of myP to be reset during the update, the parameters **reset_config** or **reset_database** in **[ADVANCED]** can be set to 1, respectively. By resetting the robot configuration or its database, you may lose important data, so be careful to only enable these options if you intend to remove your old configuration files or your database and replace them with new default files. The update will try to create a backup of the running myP version (including database and myP configuration files). If this backup fails, the update will abort and leave the robot unchanged. If you want to force the update to continue anyway in case of a backup failure, set the parameter **update_prob_force** to 1. Please note that in the worst case (if something else goes wrong too), this may lead to losing your database and configuration files. Save your changes in the file **stick-config** before you may safely remove the stick from your computer.

Update P-Rob to New Version

Make sure no application is running in myP, that the robot is in idle state (static blue front LED), that P-Rob is in a stable position (e.g. home position, standing upright) and myP is disconnected. Plug the stick into one of the USB-A ports on the connection panel of the base of P-Rob. When the robot accepts the update, the front LED will turn purple (this may take up to 30s). Once the LED is purple, do not unplug the USB stick from P-Rob until it turns blue again (this may take up to 20 min). Note that some update steps will change the LED to blue as well for some seconds before it will turn purple again! If the update succeeds, the LED will directly change to blue for the second time. If the update requires further attention, the front LED will blink red for 10s after the update finished and then turn blue again. In any case, give the robot 5s more seconds to unmount the USB stick and unplug it.

A red LED can have two reasons:

- The update requires an installation in several steps.
- A critical step of the update failed.

You find further information in the folder **update_log** on the USB stick. If you need support, save these logs for further investigation.

Please restart the robot to make sure the updated myP is running as intended.

Restore myP Backup

If myP does not work as expected after an update, the previous installation can be restored (unless the myP backup has been forced and no backup was created). To do so, in the file **stick-config** set the parameter **restore_myp** to 1 and retry the update procedure as described above. The front LED will turn purple for a short time. When it turns blue again, the old myP version has been successfully restored. If it turns red for 10s, no backup was found, or it is corrupted.

Network Setup

In order to change the network setup of your P-Rob, use the same USB stick you would use to update myP (see section **Updating**), but with different config entries, cf. below.

Set Network Configuration

Note: If your P-Rob offers several Ethernet ports, the following instructions will affect only the port labeled as **Ethernet 2**.

Open the **stick-config** file in the root directory of the USB stick. The second part of this file is dedicated to the network configuration of P-Rob. Inside you'll find the following code:

```
# Change P-Rob network configuration.
# On P-Robs of version d and newer, this changes the configuration of the 'Ethernet 2' port.
# On older P-Robs this changes the configuration of the 'Ethernet' port.
# 1: Changes the network configuration according to the settings below, 0: Does not change the
network configuration
change_ip_config = 0

# Network configuration:
# Set the port to a static IP or DHCP
# static: Use static IP, dhcp: use external DHCP server
protocol = static

# Settings for static IP configuration (not applied for protocol = dhcp)
# Note: These are example settings, ask your local network administrator for the correct settings!
ip_address = 192.168.1.203
netmask = 255.255.255.0
gateway = 192.168.1.1
dns = 192.168.1.1
```

If you connect a P-Rob directly to your computer with an Ethernet cable, you have to assign a fixed IP address to it. Specify the required network settings (**ip_address**, **netmask**, **gateway** & **network**) and set the parameter **change_ip_config** to 1. If you do not want to update or restore myP, make sure the parameter **restore_myp** and **update_myp** are set to 0! Now follow the same steps as described in the previous section. As soon as the front LED turns blue, the new settings are active. If the LED blinks red for 10s after changing the IP, the robot most probably failed to reconnect to a network. Check the log on the stick for details.

Get Network Configuration

If the robot IP is unknown, it can be retrieved using the update mechanism. Open the file **stick-config**, set the parameter **get_ip** to 1 (and disable other functions if not needed) and proceed again as described in the section **Update** . After the front LED has changed back to static blue, you can remove the USB stick and retrieve the IP from the file **IP_PROB** on that stick.

3. Desktop Interface (GUI)

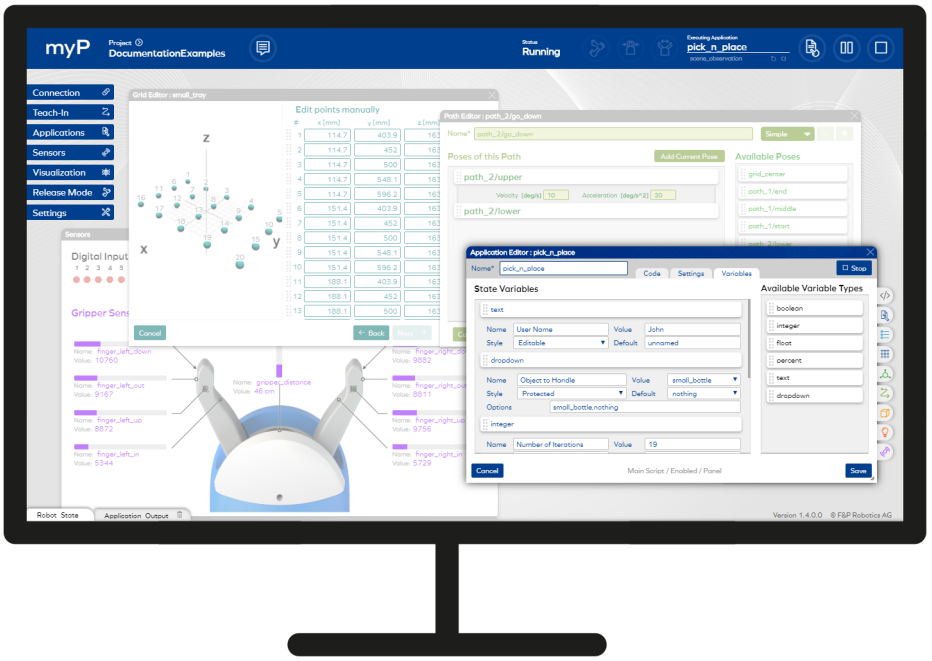


Figure 1: Example Screenshot of the myP Desktop Interface

The Desktop Interface of myP is the main Graphical User Interface (GUI) that allows for the user to control the robot, program applications, manipulate database elements, and visualize the current robot status. This GUI has been designed to be intuitive and mostly self-explanatory, even to users without much experience in robot control. This interface can be accessed via web browser (e.g. Google Chrome) by entering the IP address of the robot in the URL of that browser, which will forward you to **https://<IP>:8889/desktop**.

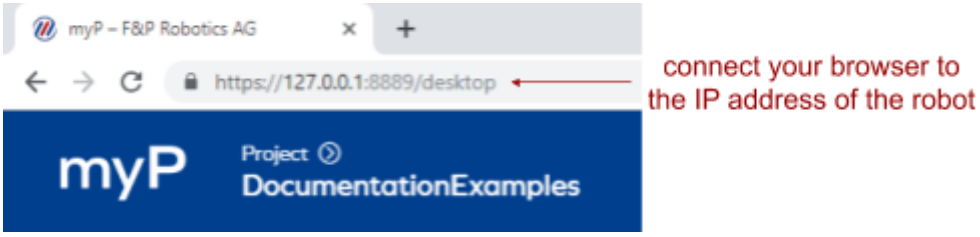


Figure 2: Connect to the Desktop Interface via URL

The Desktop Interface contains all available control functionalities for P-Rob and additional features to observe and modify the state of P-Rob. To simply execute preprogrammed applications, without the need to modify any applications or settings, we recommend to use the Panel Interface instead. In this chapter, all windows, menus, and elements of the Desktop Interface are explained in detail.

Main Window

Once you successfully connect your computer or device to the myP Desktop Interface, the main window appears in your web browser.

Figure 3: Graphical User Interface of myP

This main window contains

- a cockpit on the very top of the window, showing the current program status, messages, and quick access to the robot controls,
- the menu buttons on the right side of the window, which you can click to navigate through the available menus (which will all be explained further down in this chapter),
- tabs at the bottom of the page, showing the current coordinates and motor values, an application output console, and a list of messages logged via applications,
- the version number on the right bottom corner,
- space for message pop-ups on the right side of the screen, and
- a lot of unused space in the center where the menus appear and can be rearranged in a window explorer kind of style.

Cockpit

The cockpit of myP is always visible on top of the GUI and contains the most important information and controls. Let's explain all of the cockpit elements in detail from left to right:

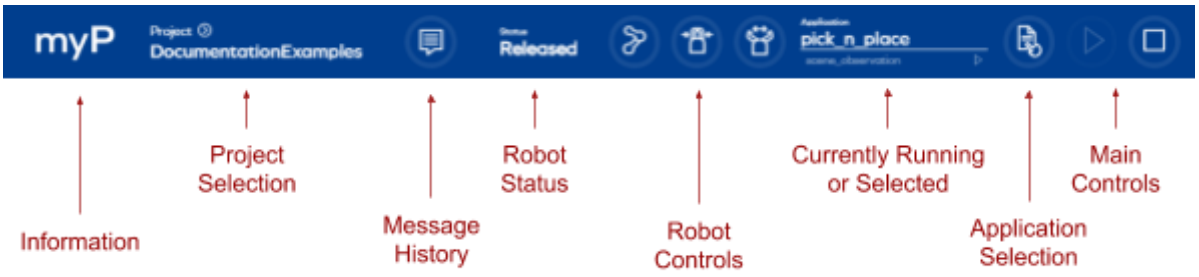


Figure 4: Cockpit

Press the **myP logo** to receive version information about myP and its connected hardware. It is recommended to always provide this information together with support requests.	
Press the **Project** button to enter the Projects Menu, where you can switch projects and see the project overview. Below this button you see the name of the active project.	
	The **Message** button opens the message history where all past messages including warnings and errors are visible in chronological order.
The **Status** message bar shows the current program status. The program status basically defines whether a command or an action such as a button press is executable or not.	
	The **Release Robot** button lets the user release all joints of the robot and guide the robot manually. The robot can only be released if the program status is ready.
	The **Hold Robot** button lets the user hold all joints of the robot. Holding the robot will change the program status back to ready.
	The **Open Gripper** button actively opens the gripper until the motion is stopped by an obstacle. The gripper can only be opened if the program status is ready.
	The **Close Gripper** button actively closes the gripper until the motion is stopped by an obstacle. The gripper can only be closed if the program status is ready.
The **Element** selection shows which element (application, path, pose, etc.) is currently linked to the control buttons (play, pause, resume, stop). This information changes whenever you click on a play button or select an application in myP	
	The **Application Selection** menu lets the user select an application that was previously created and saved into the database.
	The **Play** button executes the selected application. The program status needs to be "ready" to play an application.
	The **Pause** button pauses a running application. If an application is paused during a movement of the robot, the robot will smoothly come to a stop.
	The **Resume** button resumes a paused application. If the application has been paused during a movement, this movement will smoothly continue.
	The **Stop/Abort** button stops a running or pausing application. If an application is stopped during a movement, the robot will immediately stop and hold its position.

Information & Help

Press the **myP logo** in the top left corner to get detailed information about myP and the connected hardware, if connected. In case of support requests, please always mention the detailed version information of myP and the connected hardware. You can simply press on the “copy to clipboard” button and paste it along with your support request.

Press **F1** to open the help window showing many useful myP shortcuts to cockpit buttons and to speed up your application programming. Note that on MAC OS browsers the keyboard layout is significantly different compared to Windows or Linux OS, and therefore the list of shortcuts is different as well.

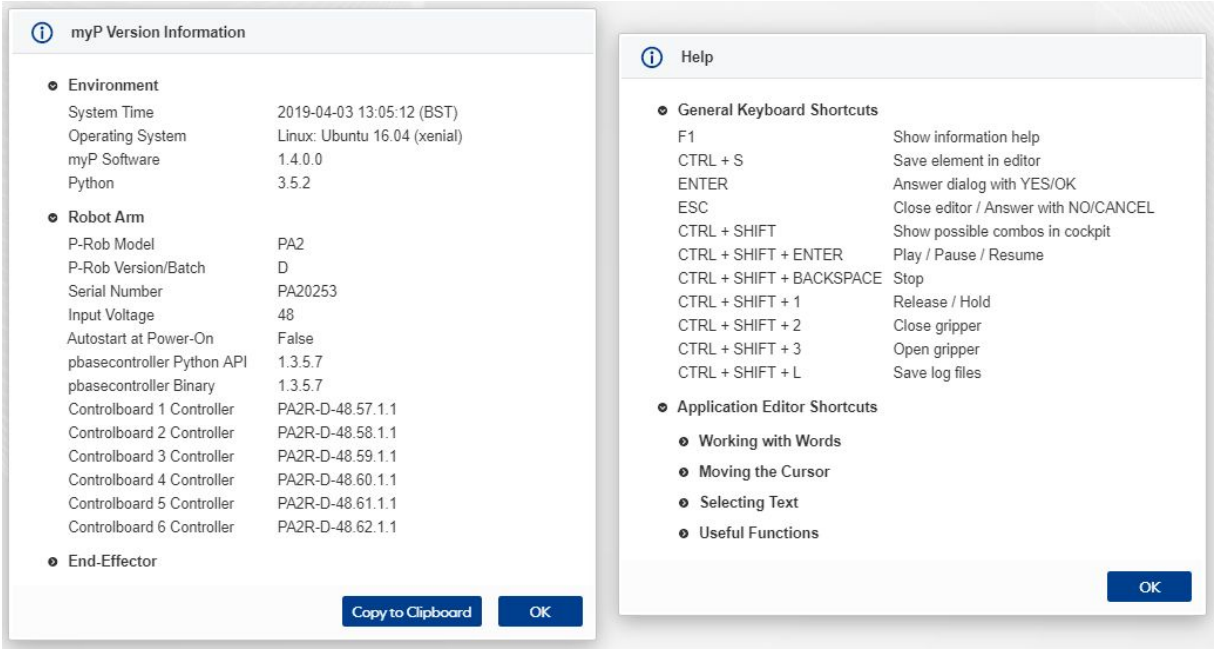


Figure 5: Version Information & Help Windows

Message History

Press the **Message History** button in the cockpit to access the messages overview window. myP differentiates between four types of messages, which are used in different contexts:

	**Information** messages provide useful messages, informative messages or hints to the user.
	**Success** messages provide feedback to let the user know that a command or action was successfully executed.
	**Warnings** provide important information to the user that must be read with caution before continuing. Warnings are never critical and do not interrupt the program flow of myP.
	There's a wide range of **Errors** in myP, from minor usability errors (e.g. wrong actions or function arguments) to critical errors caused by damaging the robot.

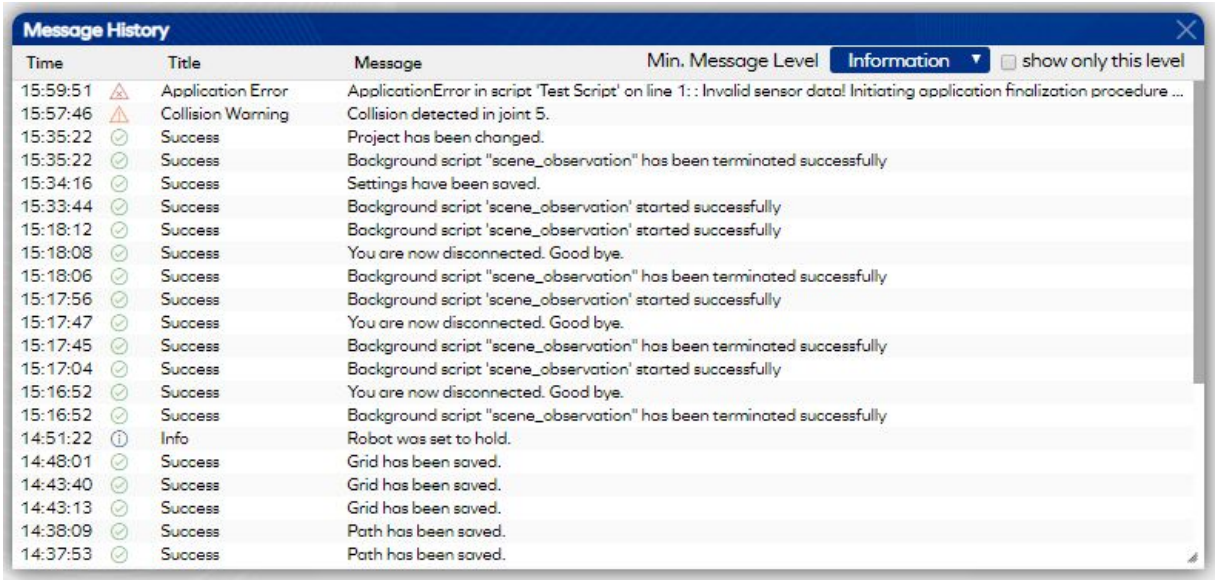


Figure 6: Message History

Program Status

The current program and/or robot status is visible in the cockpit. The status gives feedback about what myP and/or the robot is currently doing, and it basically dictates which actions or commands can currently be performed. Each status has its own properties and the availability of some functionalities of myP depend on myP being in a specific status. Here's a list of the different myP statuses and their explanations:

Connecting	The robot is currently connecting to its motors and devices. After successful connection, the robot starts actively controlling its motors and the status will change to **Ready**.
Ready	The program is ready to receive commands and/or performing actions such as executing an application, opening or closing the gripper, recording a task, etc.
Calibrating	The robot is currently calibrating. Once the calibration procedure is finished, the status will change back to **Ready**.
Running	An application/motion is currently running. This application/motion can be paused or aborted using the corresponding **Pause** or **Stop** buttons, respectively. After the application reaches the end of its code or the running motion finishes, the status goes back to **Ready**.
Paused	An application/motion is currently paused. Press the **Resume** button to continue, which sets the status back to **Running**, or press the **Stop** button to abort, which sets the status back to **Ready**.
Released	One or multiple joints of the robot are released. In this mode, you may guide the robot manually to any pose you want by applying force on its joints. Meanwhile, the robot holds its own weight by actively compensating the gravitational force. Press the **Hold** button to bring the status back to **Ready**.
Recording	The robot is currently recording something (e.g. movements or lesson samples). Press the **Stop Record** button to finish the record and set the status back to **Ready**.
Processing	A time consuming action like saving/loading data or calculating a trajectory is currently on-going. After this action has been processed successfully, the status will change back to its previous value.
Stopped	Usually, the **Stopped** status automatically recovers to **Ready** in no time. If that should not be the case, some part of myP could not be properly finalized. This unfortunately can only be recovered by restarting myP.
Error	An error has occurred that could not automatically be resolved by myP or that requires special attention from the user. Have a look at the error message and try to solve the error if possible.
Disconnecting	The robot is currently disconnecting from its motors, shutting down the motor power, and finalizing all running modules.

Menu Overview

On the left side of the main window, you will find buttons to enter the different menus of myP. The exact purpose and detailed information of each available menu is described in the following sections. Here is a quick overview over the available menus:

	The **Connection** menu lets the user connect and calibrate the robot.
	The **Teach-In** menu lets the user teach poses, paths, and grids to the robot.
	The **Application** menu lets the user write, modify, and test custom applications.
	The **Sensors** menu lets the user observe the digital I/Os and gripper sensor data.
	The **Visualization** menu displays the current motion of the robot.
	The **Release** menu lets the user release and hold specific robot joints.
	The **Settings** menu lets the user choose some basic myP settings.

Please note that buying additional myP modules like the "Learning" or the "Vision" module will enable more menus and more functions.

Robot State

When clicking on **Robot State** on the bottom of the main window, or by double-clicking on the main window, a section appears that shows the current state of the robot. This includes the coordinates of the tool center point (TCP) with respect to the base frame of P-Rob as well as the positions and currents of all motors. For a detailed explanation of the shown coordinate values, check out the Mathematical Conventions used in myP.

Application Output

Whenever an application returns text using the **print(...)** function, that text appears in the application output. Click on **Application Output** on the bottom of the main window to access this output. Note that if multiple applications are running in parallel, all of them may print text into the application output section.

Building Blocks & Database

In order to achieve easy and intuitive application programming, myP contains an object oriented structure containing different building blocks. These building blocks (further called “elements”) can be created individually in different editors which are accessible via the Desktop Interface. myP then allows the user to smartly interlink these elements to create myP applications in an object oriented way, resulting in application code that is short, easily readable, and usually does not contain many numbers and coordinates.

Here’s a list of the default elements that myP provides:

	**Applications** contain mainly script code and application specific settings. The program code supports Python 3.5 and besides basic Python functions, it also includes many functions from the myP internal robot control API (application programming interface).
	**Poses** are positions of the Tool Center Point (TCP) with specific orientations, uniquely defined by a set of joint angles. They can be created and edited in the teach-in menu.
	**Paths** are sequences of poses describing smooth movements of the TCP and therefore movements of the whole robot. They can be created and edited in the teach-in menu.
	**Grids** are customizable, up to three-dimensional arrays of points that define e.g. locations of predefined trays. Grid points are extracted and used as iterators in applications. Grids can be created and edited in the teach-in menu.
	**Sensors** provide data that can be used to integrate program logic as well as to train and detect objects. P-Grip comes with integrated sensors.

Please note that the Learning Module provides even more elements which come with additional smart functions:

	**Tasks** are a combination of specific robot skills and artificial intelligence. They are created and edited in the task generator.
	**Objects** can be grabbed and learned by the gripper. They are created and edited in the learning menu.
	**Lessons** can be used to train and detect objects using available sensors. They are created and edited in the learning menu.

All of these elements are stored in the database of myP. You can download the database of your robot at any time (to have a back-up of your complete set-up), or uploaded a new database (overwriting the current database and all of its elements) via the connection menu of this interface. Also, the projects menu provides a nice overview over all projects and elements stored in your database.

Project Menu

Click on the name of the currently selected **Project** in the title bar to access the project menu. This menu contains an overview over the most important elements available in myP, like poses, paths, tasks, objects, lessons, and applications. These elements are grouped into several projects. Only one project is selected at any time and only the elements of the selected project are visible, editable, and usable in the other menus of myP.

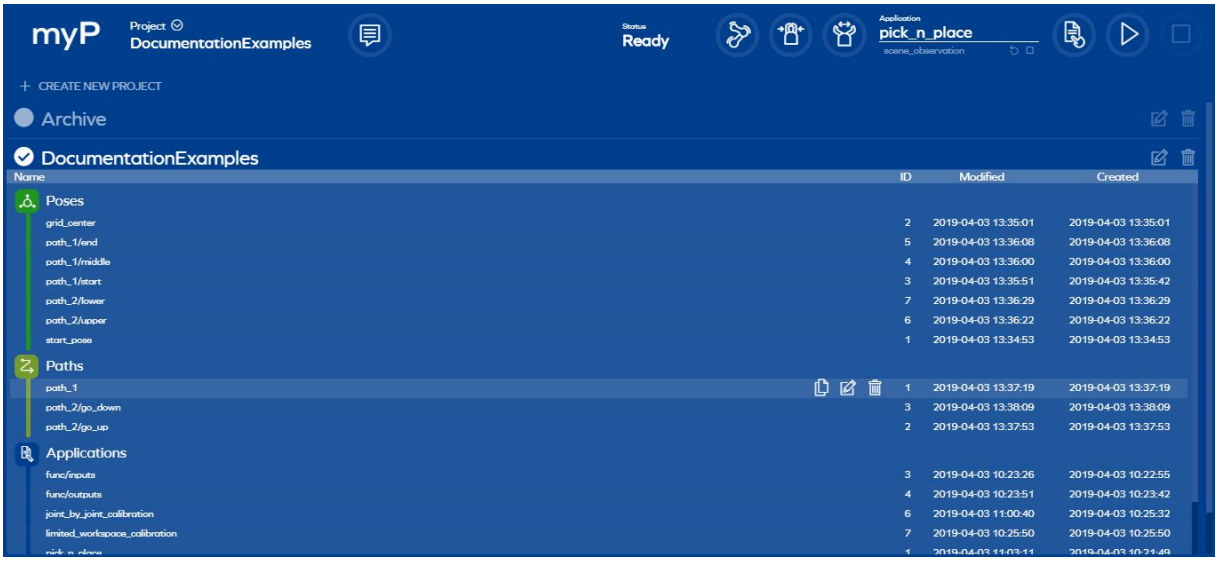


Figure 7: Project Menu

	Press the **Create New Project** button to create a new, empty project.
	Press on the circle to the left of the project name to select that project.
	Press the **Copy** button of a project element to create an exact copy of that element within the same project. The name of the copied element is slightly modified and ends with “(copy)”.
	Press the **Edit** button to the right of the project name to change the name of that project. Press the **Edit** button of a project element to open up the appropriate editor for inspection or modification.
	Press the **Delete** button to the right of the project name to remove that project incl. all of its elements. Press the **Delete** button of a project element to remove that particular element from a project.
	Press on an element symbol (e.g. Poses, Paths, etc.) to show or hide the list of elements in that particular section.
	**Move** an element from one project to another by dragging it from the source project with the mouse and dropping it in the target project. Press the **CTRL** key while dragging the element to create a **copy** of that element in the target project.

Connection Menu

Press the **Connection** button on the main page to access the connection menu.

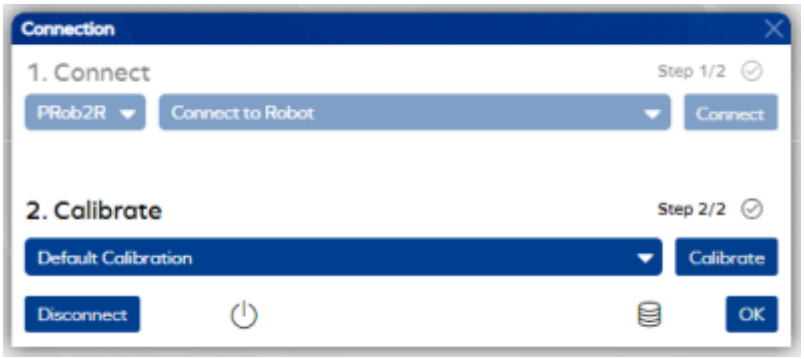


Figure 8: Connection Menu

Connecting P-Rob

To connect or disconnect P-Rob from myP, press the **Connect** or **Disconnect** button, respectively. If a check mark appears, your robot is now connected but not quite yet ready to use, because it yet needs to be calibrated. Instead of connecting to your robot, you can also connect to myP without a robot, which will basically enable all functionalities of myP that do not control the robot or request data from the hardware.

Calibrating P-Rob

Since P-Rob cannot detect its current joint angles using its internal sensors, the execution of a calibration procedure is required before controlling a P-Rob. Each time the motors of P-Rob are disconnected from their power source, the robotic arm needs to be recalibrated. The default calibration procedure moves each joint in one direction or the other. Once all motors (including end-effector actuators) have been calibrated, the robot is ready to be used.

To calibrate P-Rob, put it in an upright position, select **Default Calibration** and press **Calibrate**. Newer versions of P-Rob support limit switch calibration, meaning that the joints move until they detect an integrated limit switch and then stand up straight in the default homing position. Older versions of P-Rob move in a similar manner, though they do not include limit switches; therefore their calibration procedure moves into the mechanical stops and for safety reasons the calibration speed of those robots is reduced. If myP is used without moving the robot (e.g. only for programming applications or changing settings), the calibration step can be ignored. Please note, that for some setups or integrations of P-Rob it is not useful or convenient to use the preprogrammed default calibration procedure. For such cases, the user may define a custom calibration procedure.

Shut-down and Reboot

When hovering with the mouse cursor above the power symbol on the bottom of the connection window, three buttons appear for reboot and shut-down purposes.

	When **rebooting myP**, the software will disconnect from the hardware and myP will restart. The GUI will reload itself once myP is up-and-running again.
	When **rebooting P-Rob**, the robot will properly power down and reboot once more, changing its LED status meanwhile. Once myP restarts, the GUI will reload itself.
	When **shutting down P-Rob**, the robot will properly power down. Once the LED is off, it is safe to pull the power switch on the back-panel.

Database Download and Upload

When hovering with the mouse cursor above the database symbol on the bottom of the connection window, two buttons appear for downloading and uploading databases.

	Pressing the **Download Database** button downloads the complete database of this myP onto your local computer.
	Pressing the **Upload Database** button opens a browsing window to select a (previously downloaded) database file to upload and overwrite. Make sure to previously back-up your database if you're not sure what you are doing, since you may lose stored applications, etc. otherwise.

Teach-In Menu

Press the **Teach-In** button on the main page to access the teach-in menu. This menu contains an overview over all available poses, paths, and grids of the currently selected project.

Poses and paths can be created, edited, copied, deleted, and even directly executed. Grids on the other hand are helpful tools to keep repeating and equally spaced coordinates organized. Please note that grids are not directly used for moving a robot, but they bundle, and store sets of coordinate values in order to keep your application code structured and to prevent you from entering coordinates. A list of grid points can be extracted from a grid for diverse purposes, usually though grids are set up such that their points directly represent TCP (tool center point) coordinates, which then are used as input values for movement functions.

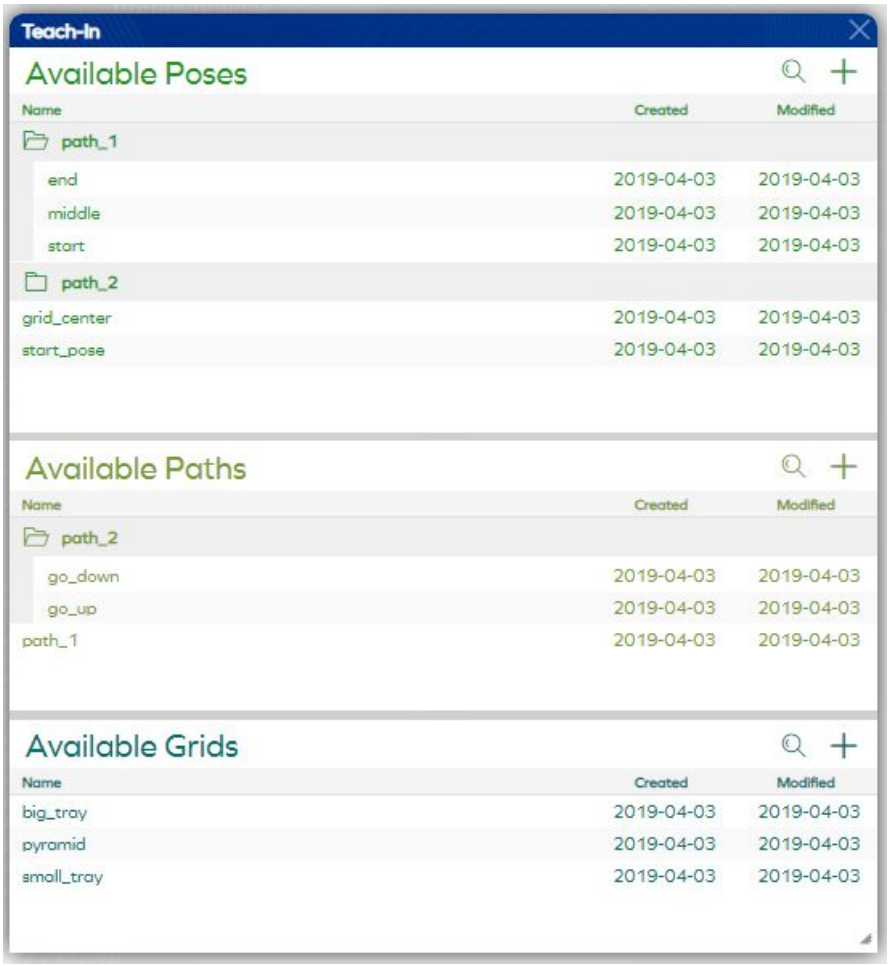


Figure 9: Teach-In Menu

	Press the **Plus/Add** button to open a pose, path, or grid editor in a new window. Further down in this section, all of these editors are explained in detail.
	Press the **Search** button to filter the list of poses, paths, or grids by name. Entering three or more characters enables the search filter.
	Press the **Play** button to move the robot to a pose or to play a path. Note that playing a path will first move the robot to the starting pose of that path, if it is not yet at this particular pose.
	Press the **Copy** button to create an exact copy of this pose or path. The name of the copied element is slightly modified and ends with “(copy)”.
	Press the **Edit** button (or double-click anywhere on the element) to open this pose, path, or grid in a separate window for inspection or modification.
	Press the **Delete** button to irreversibly remove a pose, path, or grid from your database.
	Press on a **Folder** to show or hide the poses, paths, or grids within that folder. Elements only show in folders (and subfolders) if their names contain one or more “/” characters, e.g. “folder/subfolder/name”.

Pose Editor

When creating a new pose or editing an existing pose, the pose editor opens. This editor shows the coordinate values of the Tool Center Point (TCP) pose to edit. By enabling the edit mode, the TCP is locked to the values given in the editor and the user can edit the pose by releasing the robot and manually guiding it to a custom pose or by using the arrow buttons to fine-tune this pose to a precision of ±0.1mm for positional and ±0.1° for rotational coordinates. In the following, all buttons and features of this menu are explained in detail:

Figure 10: Pose Editor

Save	By pressing the **Save** button, this pose is saved using the given name. A pose cannot be saved without a name. Use "/" characters in the name to group your poses in a folder like structure.
Cancel	By pressing the **Cancel** button, all unsaved changes made in this editor are rejected and the editor closes.
Reset	By pressing the **Reset** button, all unsaved changes made in this editor are rejected and the pose is reset to the last saved value (or the value that was given when the editor opened, if the pose was not saved at all).
Frame	By pressing on the **Coordinate Frame** drop-down menu, the user may choose in which coordinate frame to values are shown and the robot is moved using the arrow buttons. Currently available are the coordinate frames **Base** and T**ool**, meaning the coordinate frames attached to the base of the robot and the tool of the end-effector, respectively.
Show/Hide	By pressing on **Show Frame**, an image of the currently selected coordinate frame is shown. In this image, the direction of the motion effect of all the twelve arrow buttons of the pose editor is visible. By pressing on **Hide Frame**, the image is hidden once again.

Edit Mode	If the **Edit Mode** is disabled, the coordinate values shown in the editor are independent from the current pose of the robot. By enabling the edit mode, the robot is asked to move to the pose specified in the pose editor. In the edit mode, the robot is locked to the editor, meaning that when the robot is moved (e.g. by manually guiding it to a pose or by pressing the arrow buttons), the coordinate values of the pose are always updated.
Arrows	By pressing the **Arrows**, the TCP can be fine-tuned to a specific custom pose. The arrow buttons are only enabled if the edit mode is enabled, since only then the robot is locked to the values shown in the editor. Pressing an arrow button will then cause the robot to move in the direction the arrow specifies (arrow up and down meaning an increase and decrease in this coordinate, respectively) by a distance or angular difference given by the step size. If an arrow button is temporarily disabled, the robot is not able to move in that direction, because the TCP would leave its workspace.
Step Size	By pressing one of the **Position Step Size** or **Orientation Step Size** buttons, the step size of the position or orientation values change, respectively. This affects the arrow buttons, since they only increase or decrease a coordinate value by the selected step size.

Path Editor

When creating a new path or editing an existing path, the path editor opens. This editor lets you modify a path simply adding available poses by drag and drop and changing the shown parameters. The poses first need to be defined using the pose editor, otherwise they won't appear in the list of available poses. Parameters that belong to the motion from one pose to another are shown in the area between them.

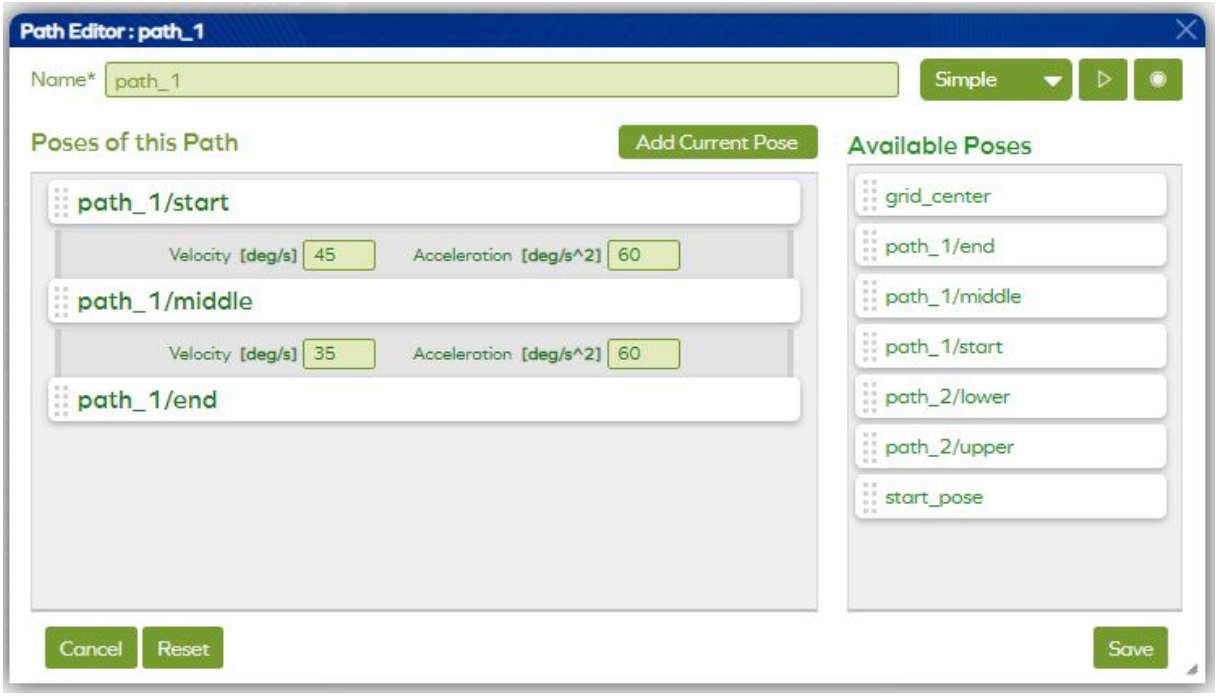


Figure 11: Path Editor

There are two different path types available: **Simple Path** and **Advanced Path**. Simple path means that myP will generate automatically the motion between poses. Velocity and acceleration can be chosen for each set of two poses. Advanced path allows you to choose which type of motion will be executed for each set of two poses. Possible options are:

- **Spline:** For this motion, a cubic spline is used to interpolate the trajectory. Use this type of motion if you want that P-Rob moves its TCP along the smooth curve between the given poses.
- **Linear:** If you choose this type of motion P-Rob will move its TCP along the straight line between the given poses.
- **Circle:** This motion will move TCP between the poses along the circle. You need to define a waypoint, located on this circle, to identify in which surface P-Rob should move.

Velocity and acceleration for advanced paths are defined globally on the top of the window. The main difference between the simple and advanced paths is, that the simple path is calculated in real-time, while the advanced path needs to be pre-calculated.

Save	By pressing the **Save** button, this path is saved using the given name. A path cannot be saved without a name. Use "/" characters in the name to group your paths in a folder like structure.
Cancel	By pressing the **Cancel** button, all unsaved changes made in this editor are rejected and the editor closes.
Reset	By pressing the **Reset** button, all unsaved changes made in this editor are rejected and the path is reset to the last saved value.
Type	By selecting either **Simple** or **Advanced** from the drop-down menu, you can choose the mode of the path. Both path modes work differently and allow for different path parameters to be tuned.
Add Pose	To use a predefined pose inside a path, drag and drop one of the available poses into the **Poses of this Path** section. Alternatively, by pressing the **Add Current Pose** button, the current pose of the robot is measured and added to the very end of the pose list.
Play	By pressing the **Play** button on top of the window, the robot will execute this path. If the current pose of the robot does not match the starting pose of the path to play, the robot will first move to this pose first. Press the **Play** button of a pose to move to that particular pose.
Record	By pressing the **Record** button, the robot will release its joints and record its movements until the recording is stopped or interrupted.
Remove	By pressing the **Delete** button of a pose that has been added to the path, this pose (and the following path segment) is deleted from the path.
Edit	By pressing the **Edit** button belonging to one of the available poses, the pose editor is shown in a new window to edit and fine-tune this pose.

Grid Editor

When creating a new grid, the grid editor opens. This editor guides the user step-by-step through all the available grid options. Use the buttons **Next** and **Back** to navigate through this wizard, until the desired ordered list of grid points is defined as desired. By editing already existing grids, the grid editor jumps directly to the manual point editing (step 4).

In a first step, the user must choose the **Grid Dimension**. Usually, a grid is interpreted as a two-dimensional array, though it may as well be a simple list of points without depending on a second dimension (e.g. for picking objects from a conveyor belt), or even a three-dimensional structure (e.g. stacking objects in boxes). The grid is initialized with perfectly aligned points, though note that single grid points may always be modified, added, or removed later on.

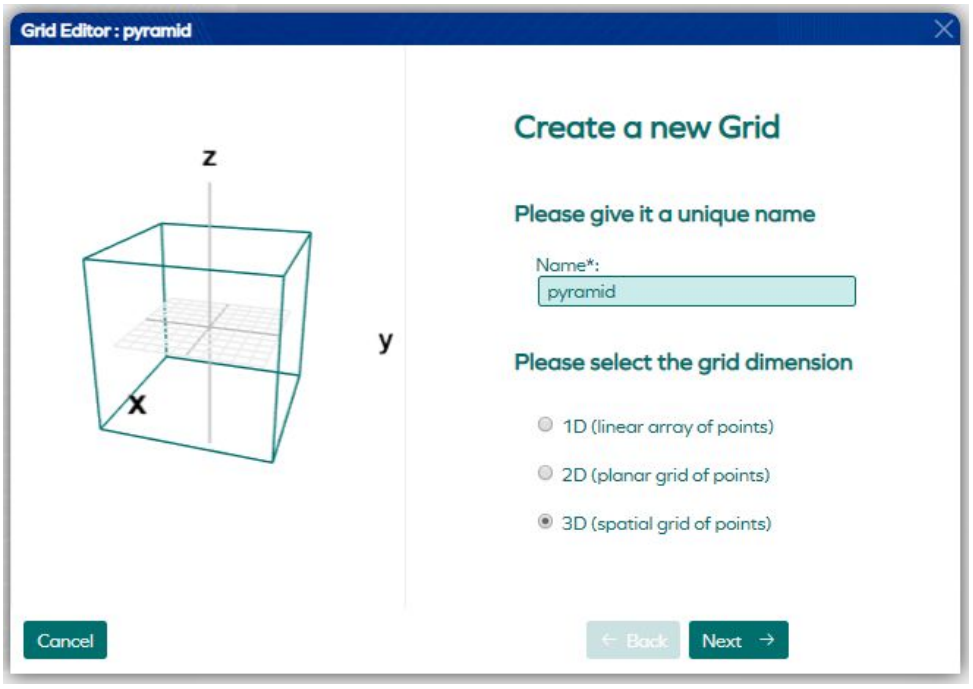


Figure 12: Grid Editor (1st step)

In a second step, the user must choose the **Corner Points** of the selected grid. Grids of different dimensions have a different number of corners, and the corners are labelled from “A” to e.g. “D” for a two-dimensional grid. The user may enter the corner coordinate values (given in the base frame of the robot) directly in the input fields, or may simply guide the TCP of the end-effector to the corners of the grid and press the appropriate “teach” button to save the coordinates.

Once enough corners are taught to define the grid (e.g. at least 3 points for the two-dimensional grid), the user may press the **Evaluate Values** button to evaluate the given corners and auto-fill the missing values to fit them to a parallelogram (two-dimensional) or a parallelepiped (three-dimensional). With the additional **Auto-Alignment** flag activated, the evaluation procedure takes into account horizontal and vertical coordinate axes and planes and tries to align the resulting grid along these axes and planes (if close enough).

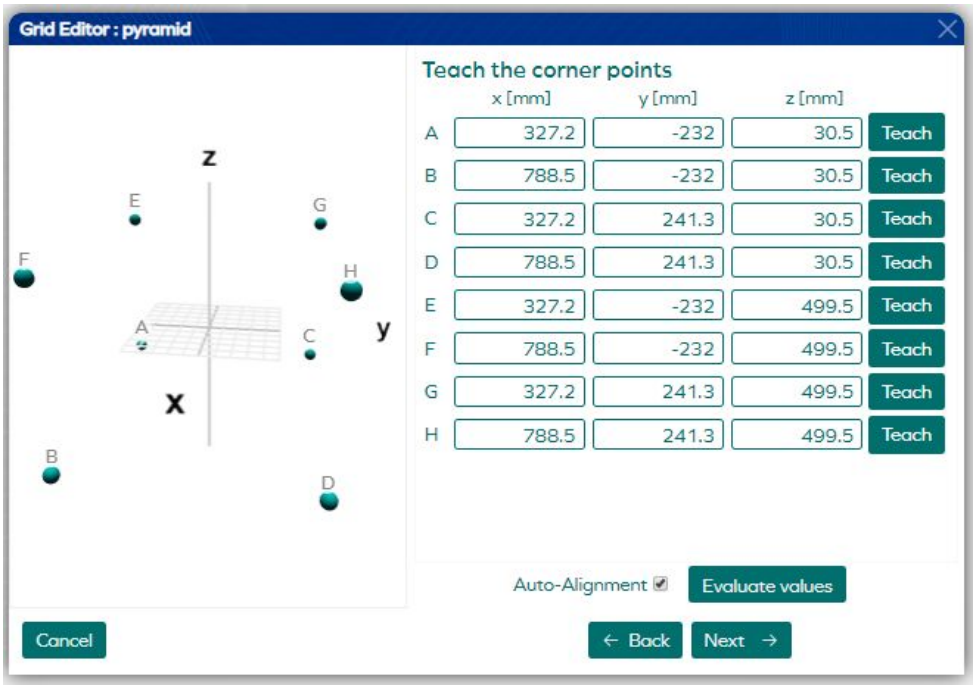


Figure 13: Grid Editor (2nd step)

In a third step, the user is asked to choose the number of points, special extension options and iteration directions. Let's have a look at these available options in detail:

- The **Number of Points** can be set for each dimension individually. If you find it hard to distinguish between the dimensions (e.g. rows and columns), refer to the labels of the previously taught corner points.
- For two-dimensional grids, there exists an **Extension Option** for adding intermediate points between each set of four neighboring points. For three-dimensional grids this option exists for each grid plane as well as for the set of eight neighboring points in space.
- The **Iteration Direction**, meaning the direction in which the points are returned and used as iterators in programming loops, is given by a starting point and a vector in each direction. The last direction is always given by the previously selected options and is simply shown for reference and visual verification. The letters in the drop-down menus correspond with the labels of the corner points of that grid.

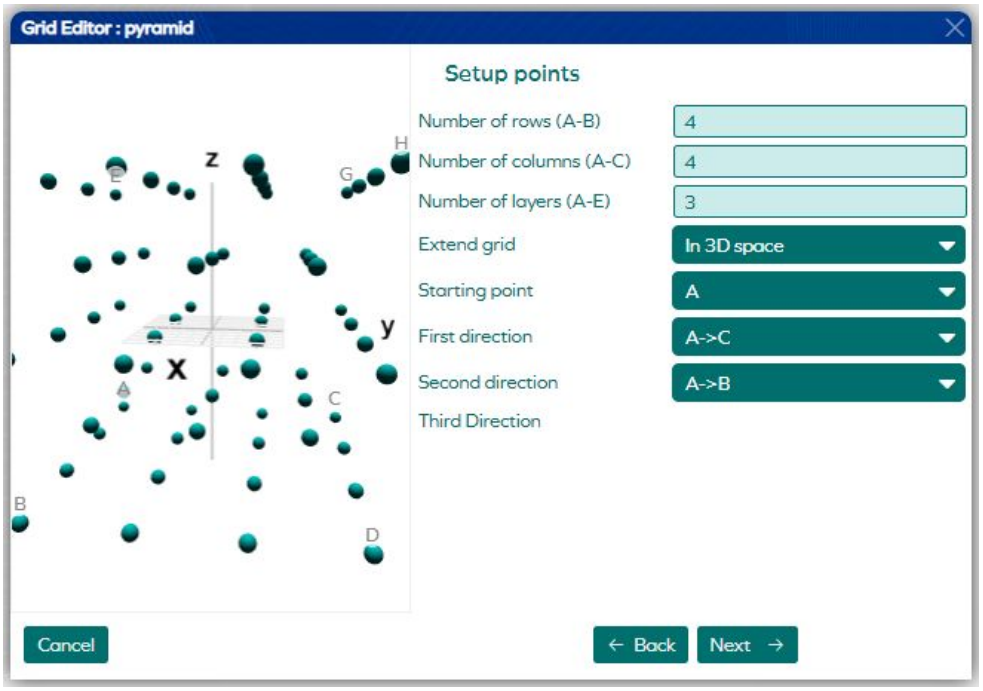


Figure 14: Grid Editor (3rd step)

Finally, the **List of Grid Points** is ready for manual editing. Hereby, single points can be copied, deleted, moved, and coordinate values may be manually changed. Meanwhile, the visualization represents the point order and the relative coordinates of the grid points.

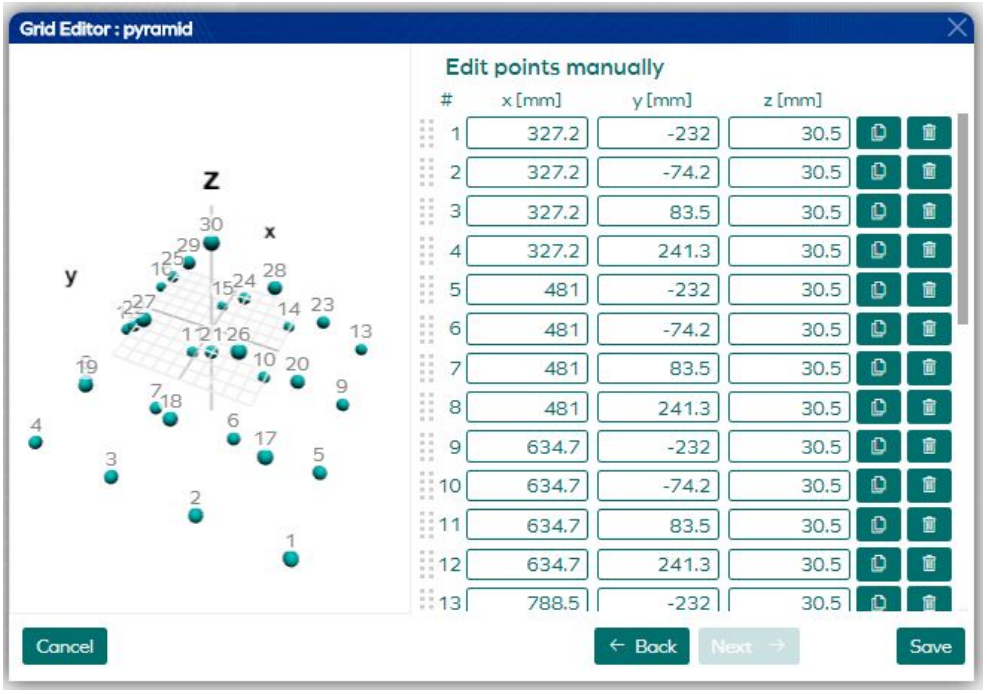


Figure 15: Grid Editor (4th step)

While editing, you may always go back to previous wizard windows to inspect your previously set options and values. You may also change those options, but keep in mind that changes in one window will reset all changes in the following windows.

Application Menu

Press the **Application** button on the main page to access the applications menu. This menu contains an overview of all saved applications as well as the output of the last executed application. It also allows the user to access the **Application Editor** window to create new or edit existing applications.

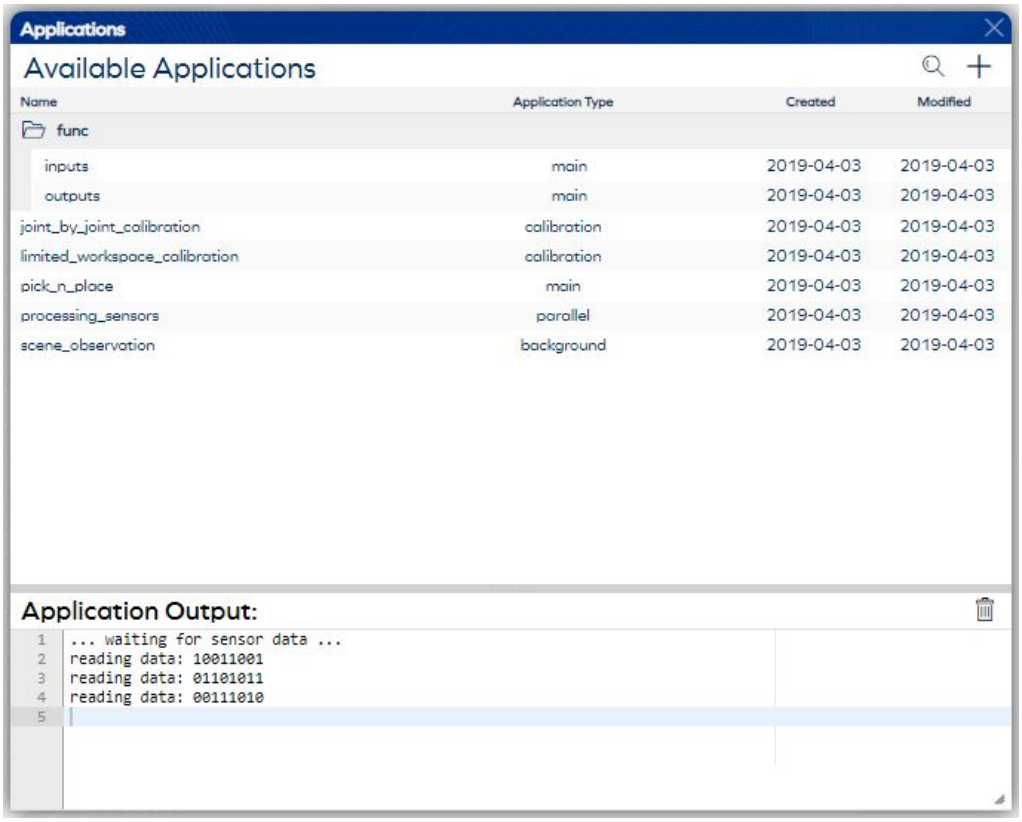


Figure 16: Applications Menu

	Press the **Plus/Add** button to create a new application. This opens up the application editor, whose features are explained in detail further down in this section.
	Press the **Search** button to filter the list of applications by name. Entering three or more characters enables the search filter.
	Press the **Play** button to run an application. Note that only “main” applications are executable this way, as only “main” applications are selectable in the cockpit.
	Press the **Copy** button to create an exact copy of this application. The name of the copied element is slightly modified and ends with “(copy)”.
	Press the **Edit** button (or double-click anywhere on the element) to open this application in a separate window for inspection or modification.
	Press the **Delete** button to irreversibly remove an application from your database. This removes the program code as well as the application specific settings.
	Press on a **Folder** to show or hide the applications within that folder. Elements only show in folders (and subfolders) if their names contain one or more “/” characters, e.g. “folder/subfolder/name”.

Application Editor

The **Available Applications** section of the **Applications** menu lists all previously saved applications. You may modify an existing application by clicking the **Edit** button or create a new application by pressing the **Create New Application** button. In both cases the **Application Editor** window will pop-up. Its functionality is similar for both cases. Let's quickly have a look at the main buttons of the application editor, before going into detail:

Save	By pressing the **Save** button, the application is saved using the given name. An application cannot be saved without a name. Use "/" characters in the name to group your applications in a folder like structure.
Cancel	By pressing the **Cancel** button, all unsaved changes made in this editor are rejected and the editor closes.
▷ **Test**	By pressing the **Test Application** button on top of the window, the application runs in test mode. Test mode means that the script code executes but does not prevent you from continuing editing that application. Changes in the application that have been added after testing an application are not executed.

Application Code

The application **Code** tab is the main one of the three sub-windows of the application editor. This section shows the application code and optionally the library with myP functions and myP elements to insert in the application code. The programming language that is integrated into myP is **Python 3.5**, with most of its basic functionality, including some pre-installed Python packages. In addition to that, myP comes with a big library of functions designed to simplify the writing of a robotic application (e.g. moving the robot, reading sensor data, communicating with the user, etc.). For details about these functions, please refer to the Script Functions Manual.

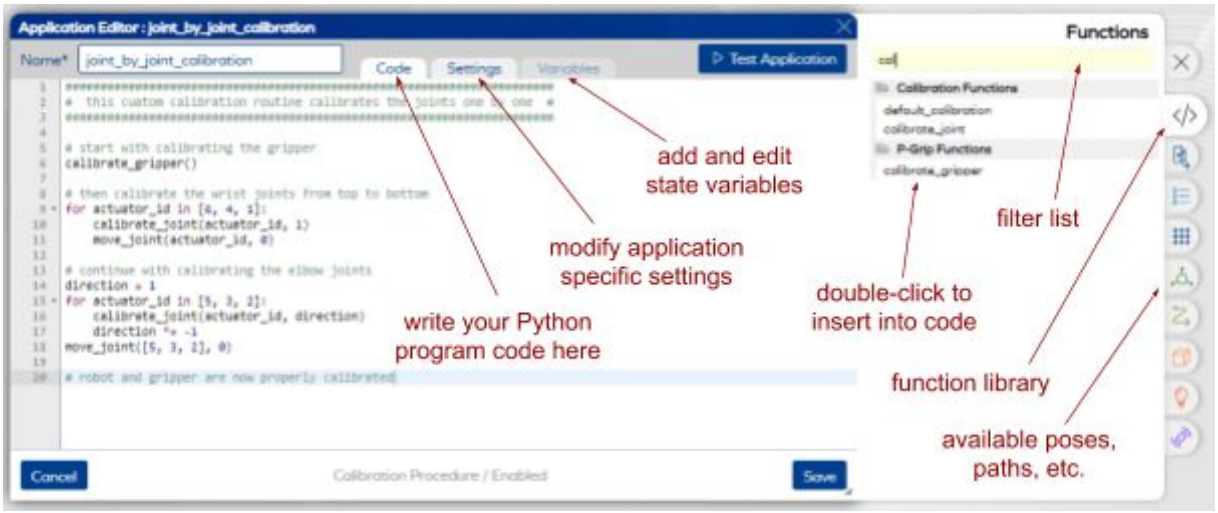


Figure 17: Application Editor (Code Section)

Press on one of the **Function Symbols** or one of the **Element Symbols** on the right side of the editor to open up the element **Palette**. Here you will find a full list of available functions, including myP core functions, as well as additional functions in case you installed additional software add-ons on your P-Rob. Also, you have access to all elements that you can integrate in your code by double-clicking on them, so you don't have to remember the names of your elements.

Hint: Increase the readability of Python code by removing the optional arguments that are inserted into your code by double-clicking on a function. You may e.g. write

```
open_gripper(10)
```

instead of

```
open_gripper(position=10, velocity=None, acceleration=None)
```

Application Settings

The application **Settings** tab contains some additional settings for the application, including

- an optional **Description** text (which in this version is temporarily used to define script labels, meaning translations of the script name into one or multiple languages),
- an **Executable** flag that can be disabled e.g. if your application is still under construction and should not be executed,
- a flag to **Show** this application in the Panel Interface,
- functionality to upload and replace the **Picture** shown in the Panel Interface,
- and most importantly the **Application Type**, which is explained in the following:

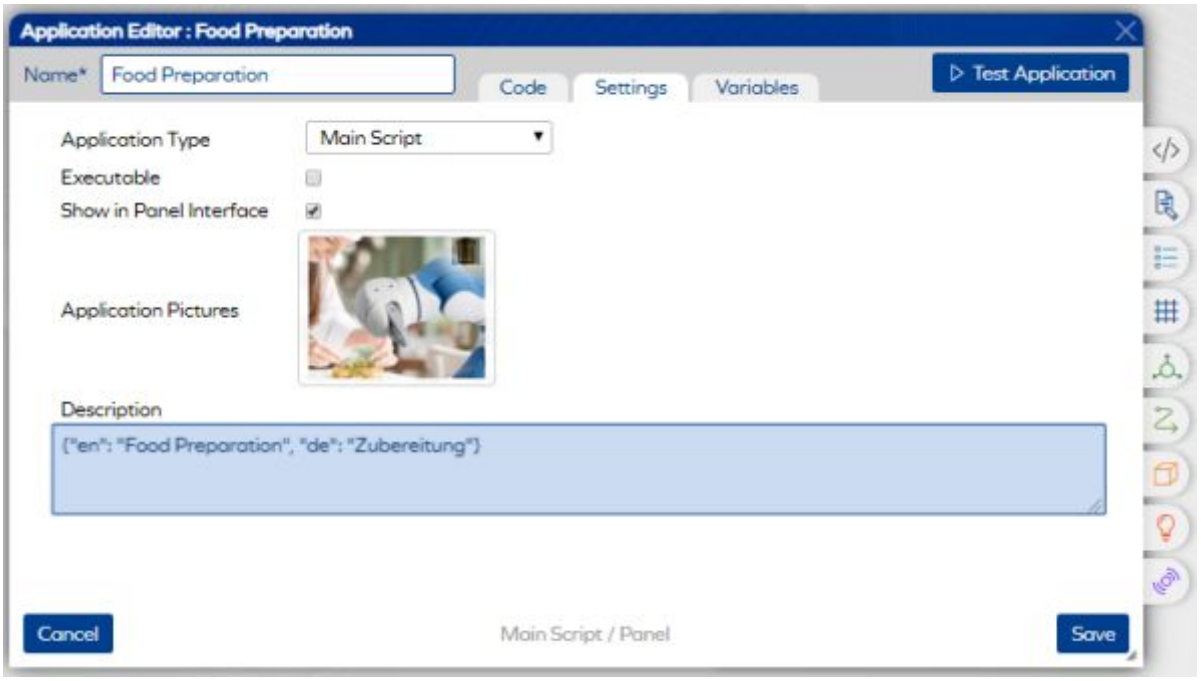


Figure 18: Application Editor (Settings Section)

The **Application Type** specifies in which context the application is supposed to be run, and therefore dictates how an application can be executed as well as which functions can be used within that application:

- **Main Scripts** contain the main applications to be executed. They are designed to use all the available script functions and to directly control the robot as well as its external devices. Usually, a simple application can be written in a single script, but for long or complicated applications, or to improve readability, it can be useful to split up an application into multiple subscripts that run sequentially to one another. Multiple main scripts never run in parallel to one another though, for this purpose parallel scripts are needed. You may also use the “import” statement to import functions across Main Scripts. Note that only Main Scripts are selectable from the cockpit.

- **Parallel Scripts** on the other hand run in separate threads and therefore are designed to run in parallel to a main application. This can be very useful if it comes to e.g. continuously reading sensor data, using the idle time of the robot moving to compute power consuming calculations. Parallel scripts are not allowed to use all script functions, like e.g. controlling the robot, since this could seriously mess up the program flow or invoke unwanted movement behaviour if not paying close attention. All parallel scripts are terminated once the main application stops running.

- **Calibration Scripts** allow the user of myP to define custom calibration procedures instead of using the default calibration routine. This can be very useful if e.g. the workspace of the robot is limited in a way that the default calibration would fail, or if the robot is to be directly moved to a different homing position after its calibration procedure. For details on calibration scripts, refer to the chapter Custom Calibration.

- **Background Scripts** are scripts that run in the background of myP, independently on its status, and are therefore useful for e.g. constantly observing, checking, or logging data, or to define initial global variables for a project. Each project can contain only one background script. If this one is defined, it will automatically execute once the appropriate project is loaded (e.g. when starting up myP or when changing a project). The background script does not listen to the status restrictions of myP and – if containing a loop – may run until myP is disconnected or P-Rob is shut down. Whenever a background script starts and finishes, an appropriate notification is displayed. You may start, restart, or stop the background script from the cockpit.

Figure 19: Background Script Controls

Application Variables

The application **Variables** tab contains a drag and drop interface to add different types of state variables to this application. State variables are used in the Panel Interface, where they are very useful to e.g. monitor the state and progress of an application or to specify certain input values to an application. Use the functions **set_state_variable(...)** and **get_state_variable(...)** to write and read state variables, respectively.

Here's an overview of the variable types and how they are displayed in the Panel Interface:

boolean	Booleans are represented as simple on-off sliders that are either enabled or disabled. They are read and written as Python type **bool**, where enabled matches True and disabled matches False.
integer	Integers are represented as text fields that allow numerical inputs, including +/- buttons to increase and decrease their values in edit mode. They are read and written as Python type **int**.
float	Floats are represented as text fields that allow numerical inputs in edit mode. They are read and written as Python type **float**.
percent	Percent variables are represented as sliders in edit mode, with a mandatory minimum and maximum value. They are read and written as Python type **float**.
text	Text variables are represented as text fields in edit mode. They are read and written as Python type **str**.
dropdown	Dropdown variables are represented as dropdown lists in edit mode. They are read and written as Python type **str**.

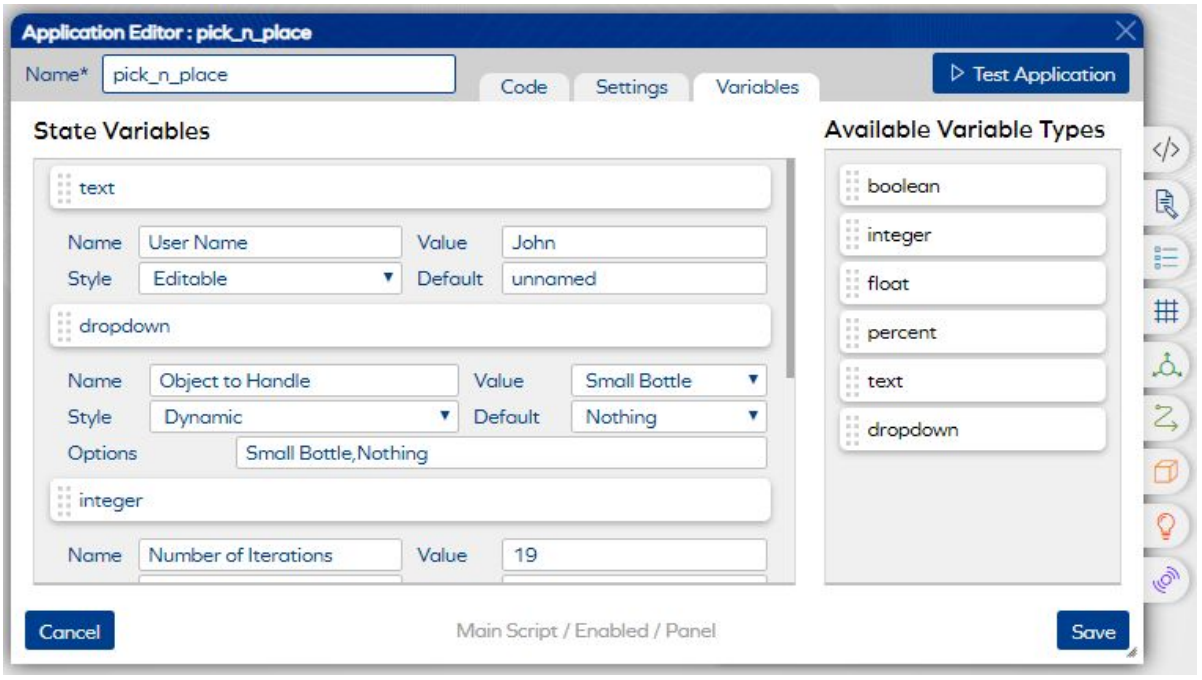


Figure 20: Application Editor (Variables Section)

Here's a quick overview over the variable options, which vary depending on the variable type:

Name	Give your variable a suitable name, it will later be accessible by that name in string form.
Value	The value of a variable contains the currently set value for that variable, that will be read and written in the Python equivalent type explained above.
Default	The default of a variable is the value that it takes when resetting the variables using the according button in the Panel Interface.
Style	The style of a variable defines if, when, and how this variable is editable.
Min	Numerical value types allow for setting a minimum boundary on that number. If you try to enter values lower than the minimum, the minimum will be taken. Boundaries are mandatory for percent variables and optional for integers/floats.
Max	Numerical value types allow for setting a maximum boundary on that number. If you try to enter values higher than the maximum, the maximum will be taken. Boundaries are mandatory for percent variables and optional for integers/floats.
Unit	For numerical value types, you may add an optional unit string to the number.
Options	The options for dropdown variables can be entered in a comma separated string.

Here's a quick overview over the variable styles, which define how a variable is used:

- **Editable** variables allow for changing the value before running the application, but not during run-time.
- **Protected** variables are editable, and as a protection layer you have to press on an edit button to modify the value and on a save button to accept the value change.
- **Dynamic** variables are editable, and you may even dynamically change their values on run-time.
- **Dynamic & Protected** variables share the features of protected and dynamic variables and are therefore editable on run-time by pressing on an edit button.
- **Read Only** variables are not editable, their value can only be changed by the application. They are usually used to monitor the state or progress of an application.
- **Hidden** variables are not shown in the Panel Interface, but can be written and read the same way as other state variables.

Shared & Global Variables

In addition to the application specific state variables explained above, shared and global variables can be defined to share data between multiple in parallel running scripts or multiple sessions of the same application:

- **Shared Variables** exist only during run-time of a single application. Use the functions **set_shared_variable(...)** and **get_shared_variable(...)** to share values amongst multiple scripts running in parallel or sequence in that application.
- **Global Variables** share values not only between the scripts of an application, but between all applications and still exist even after rebooting the robot. Use the functions **set_global_variable(...)** and **get_global_variable(...)** therefore.

Sensors Menu

Press the **Sensors** button on the main page to access the sensors menu. This menu contains a graphical representation of all the sensors, inputs and outputs connected to myP.

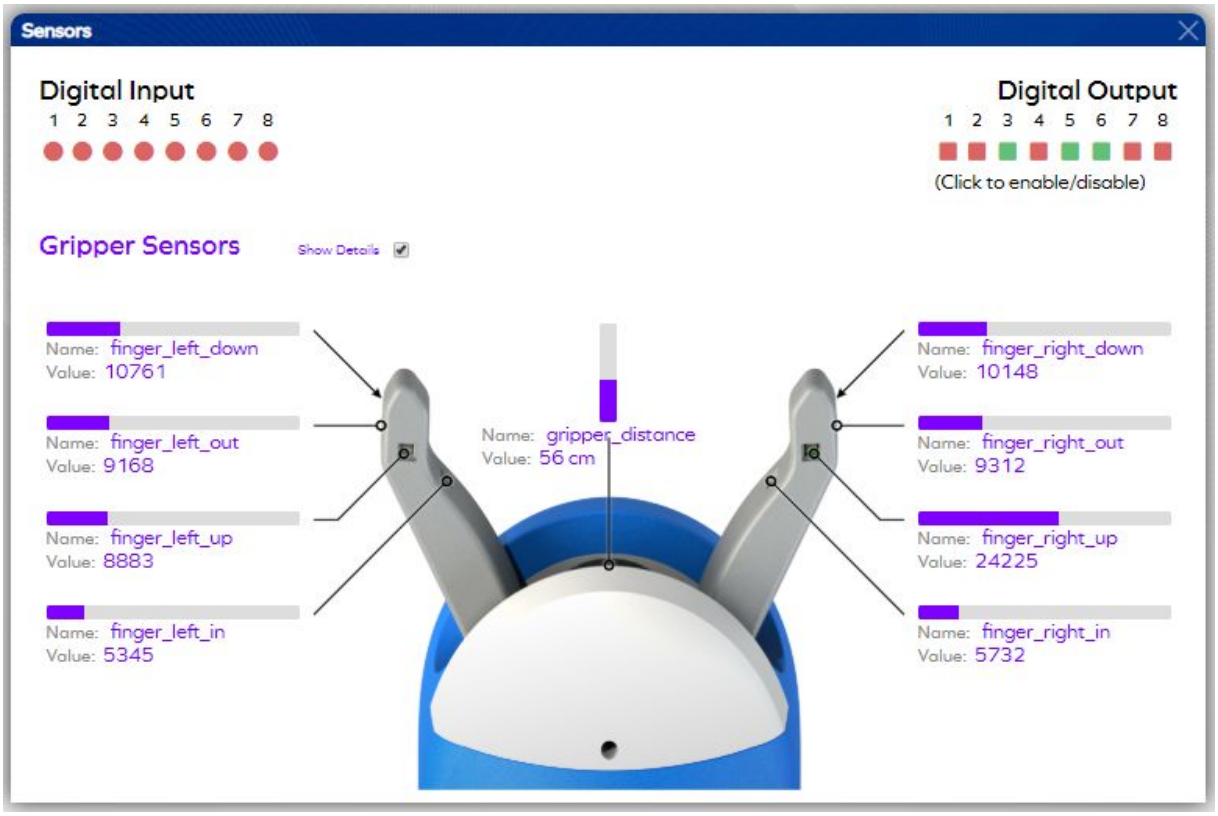


Figure 21: Sensors Menu

The upper part of this menu shows the status of the (8-bit) **Digital Inputs and Outputs** of P-Rob. Digital inputs and outputs are most commonly used to receive events from and send events to cooperating machinery, respectively. Digital I/Os can also be used to e.g. synchronize P-Rob with other machinery or to simply control the behaviour of other machinery via myP. For a detailed explanation of the digital I/O connectors on the back panel of P-Rob and how to power them properly, please refer to the P-Rob User Manual.

The lower part of this menu shows the locations, names and current values of the connected **Gripper and Finger Sensors** of P-Grip. It is also visible if single sensors lose their connection to myP. For a detailed explanation of the gripper and finger sensors, please refer to the P-Grip User Manual.

Settings Menu

In the **Settings Menu**, the user permanently changes some of the basic settings of myP. Press **Save** to accept all changes, press **Cancel** to reject them, or press **Reset** to reset all settings to the latest saved values. In the following, all settings categories and options are explained:

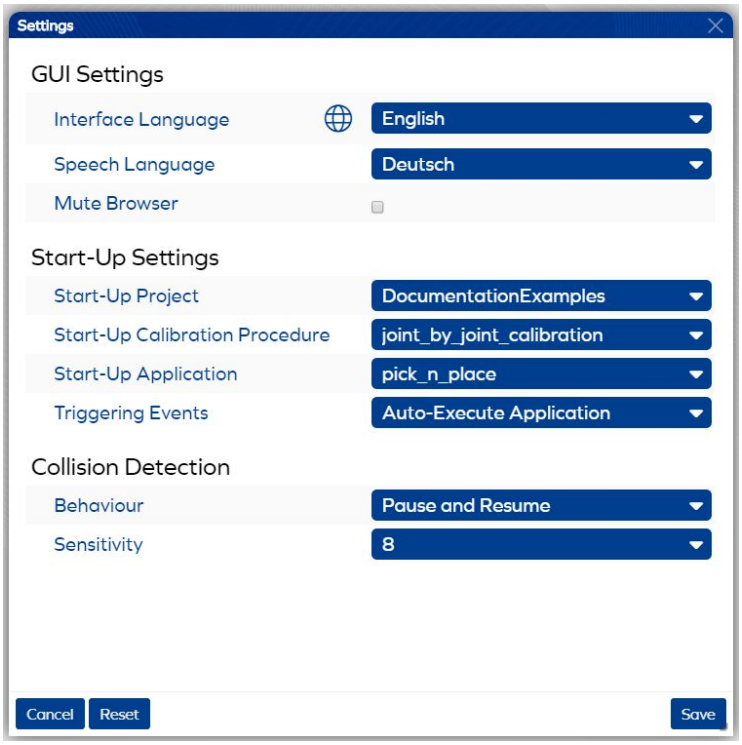


Figure 22: Settings Menu

Language Settings

This category contains all language related settings.

Interface Language	Choose the language for all built-in text shown in the myP interfaces. Currently available languages are English, German, French, Italian, Spanish, and Chinese. Note that the names of your projects, applications, variables, and other myP elements are not translated, and basically can be chosen to be in any language.
Speech Language	Choose the default language for speech outputs via open myP browser windows.
Mute Browser	Enable this flag to mute the browser. Speech outputs will no longer be played via open myP browser windows.

Start-Up Settings

In this category, all settings refer to the start-up procedure of myP. Modifying these settings helps to avoid repetitive actions upon myP start-up. Choose the list entry **Last Selected** (default entry) for some start-up settings for myP to remember and reload its last session.

Start-Up Project	Select the project that is loaded at myP start-up. All defined projects appear in this list.
Start-Up Calibration	In case a project is selected, choose the calibration routine that is loaded at myP start-up. All predefined calibration routines as well as the calibration scripts of the selected project appear in this list.
Start-Up Application	In case a project is selected, choose the application that is loaded at myP start-up. All defined main applications of that project appear in this list.
Triggering Events	Choose, if you want myP and P-Rob to automatically connect, calibrate, or even execute an application at start-up. By default, this option is disabled. In order to select a trigger, some of the start-up settings listed above have to be selected first, to prevent the user from accidentally executing unintended calibration procedures or applications.

Collision Detection Settings

In this category, all settings relate to the collision detection feature of P-Rob. A collision is detected, whenever the currents in the joints of the robot do not match the dynamic model of the robot anymore, to a certain degree.

Behaviour	Choose the behaviour of myP and P-Rob upon detecting a collision. The available options are: ● **Pause and Resume** causes the application to pause and automatically resume after five seconds. ● **Pause and Approve** causes the application to pause and myP waits for approval by the user (dialog window) to continue or stop the application. ● **Stop and Error** causes the application to stop and throw an error message. ● **Stop and Release** causes the application to stop and myP to enable the gravity-compensation mode for manual guidance. ● **Raise Interrupt** causes a “CollisionInterrupt” to be raised inside the application code. This allows us to define custom interrupt scenarios, or raises a usual error if the interrupt is not caught.
Sensitivity	Choose the collision detection sensitivity for P-Rob. This value specifies whether it takes small or large external forces acting on the robot to trigger a collision. High sensitivity values reflect a high sensitivity to external forces and in some cases, may even cause a collision to trigger when no external forces act. Low sensitivity values on the other hand increase the threshold for triggering collisions. Use low sensitivity values with caution, since they may cause a collision to be triggered too late and may in the worst-case result in damaged motors. If the sensitivity has been changed to different values for different joints, **Custom Values** is selected.

4. Panel Interface (GUI)

Figure 23: Example Screenshot of the Panel Interface

The panel interface of myP is an alternative to using the Desktop Interface to quickly access your main applications. This interface has been designed to be fully adaptable to customer needs and especially to only display the applications (and information about their state). The panel interface is touch screen compatible and its design resembles the well-known app-like structure of state-of-the-art mobile devices, making it intuitive even for users without technical background. Also, using the panel interface, the user does not have to care about programming or the code behind the application. Just click on the appropriate app and the application to execute is selected. The panel interface can be accessed via web browser (e.g. Google Chrome) by entering **https://<IP>:8889/panel** into the URL of the web browser.

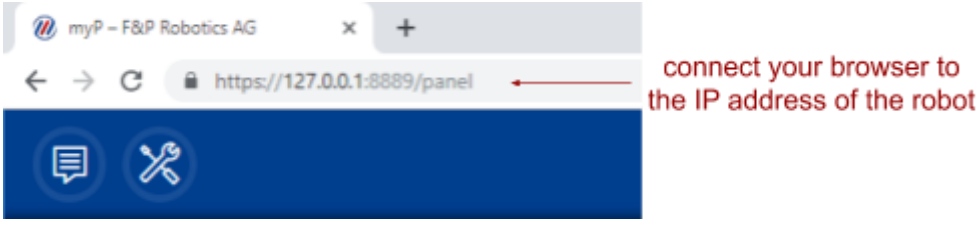


Figure 24: Connect to the Panel Interface via URL

Configure Applications for Panel Interface

By default, the panel view of myP is empty, meaning that no applications are selected for quick execution. In order to configure an application that you want to display in the Panel Interface, follow these steps:

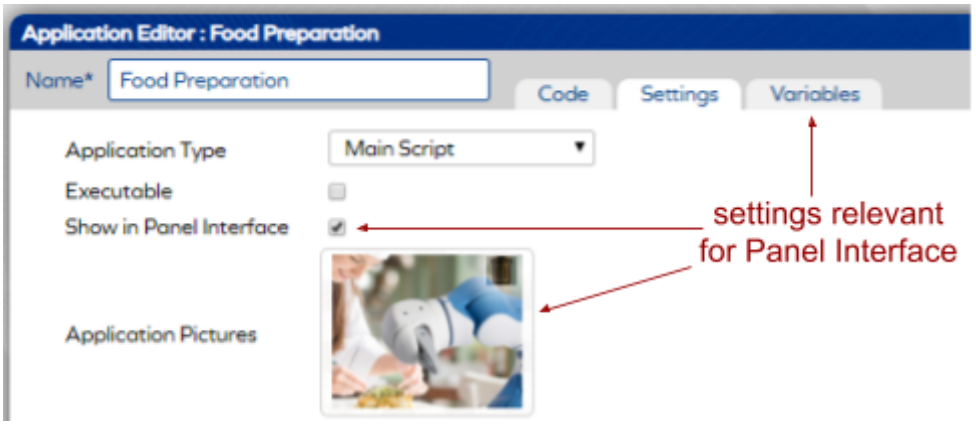


Figure 25: Panel Interface Relevant Settings in the Application Editor (Desktop Interface)

- navigate to the Desktop Interface, edit the application, go to the Application Settings,
- enable the **Show in Panel Interface** flag,
- optionally upload an appropriate **Application Picture** (the optimal size ratio is 5:4), which later shows together with the name of the application in the main menu of the Panel Interface,
- optionally navigate to the Application Variables tab to add **State Variables**, which allows you to monitor and/or modify variables via the Panel Interface,
- and finally **Save** the application.

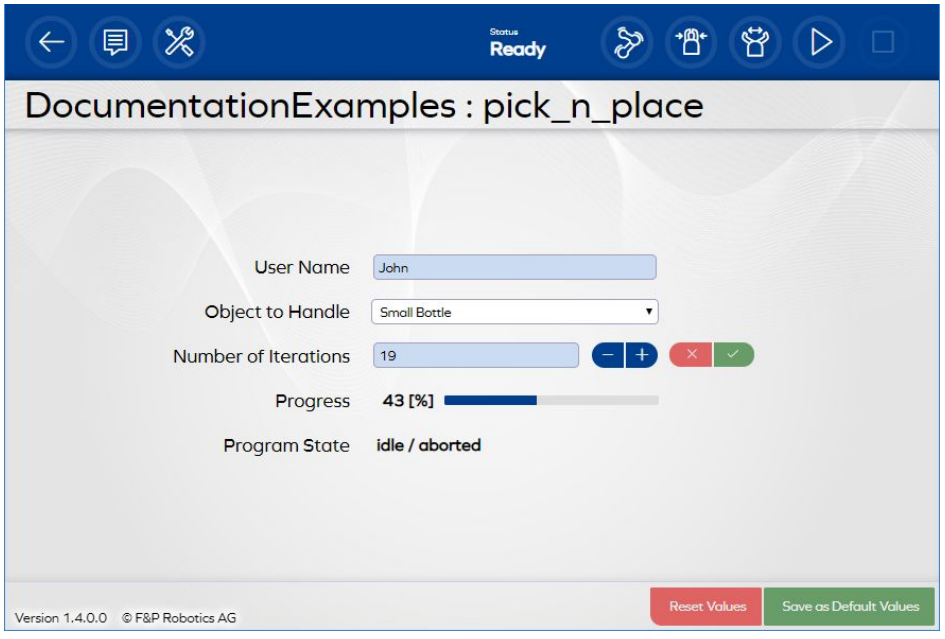


Figure 26: Panel Application with State Variables

Cockpit and Settings

The cockpit of the Panel Interface shows more or less the same buttons as the cockpit of the Desktop Interface. Please refer to the section Cockpit for a detailed explanation of all the elements. In addition to these elements, you will see two new buttons here:

	The **Settings** button leads you to the settings window, which contains only the most important settings for quick-access, namely ● connecting and disconnecting myP, ● calibrating the robot, and ● changing the interface language.
	Press the **Back** button to go back to the main window of the Panel Interface.

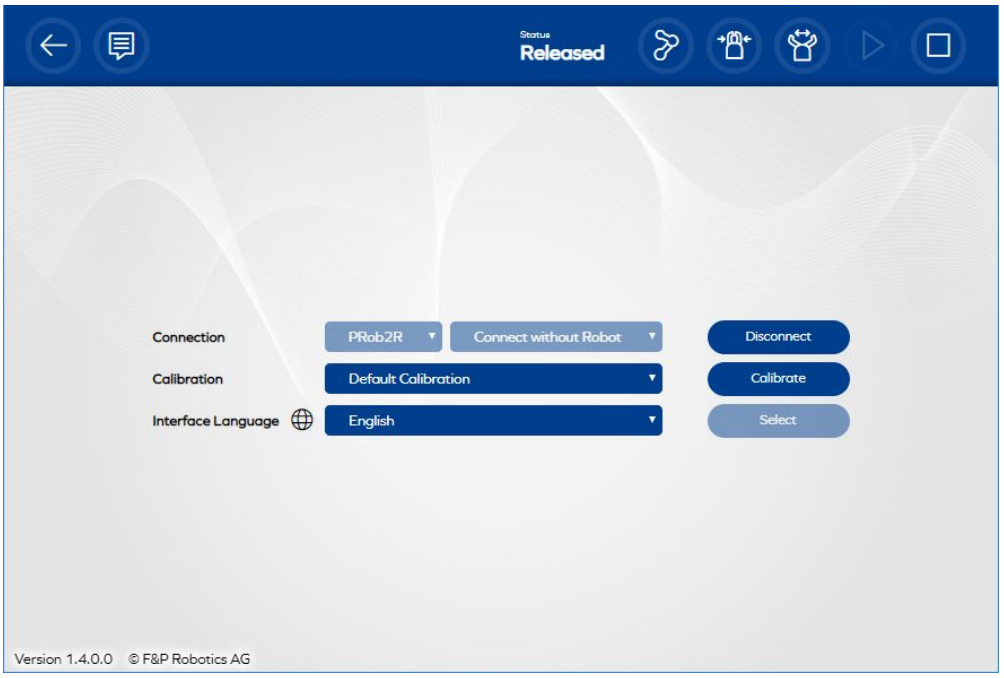


Figure 27: Panel Settings Window

Controlling Applications

In the main window of the Panel Interface all applications are listed in an app-like structure, showing their name and application picture. The panel differentiates between to kind of applications:

- Applications **without state variables** show a play button. Press on the application button to directly run this application.
- Applications **containing state variables** show without a play button. Press on the application button to enter the application detail view showing all state variables. In this window you can change the editable state variables and press the play button to execute this application. Press the back button to return to the main panel window.

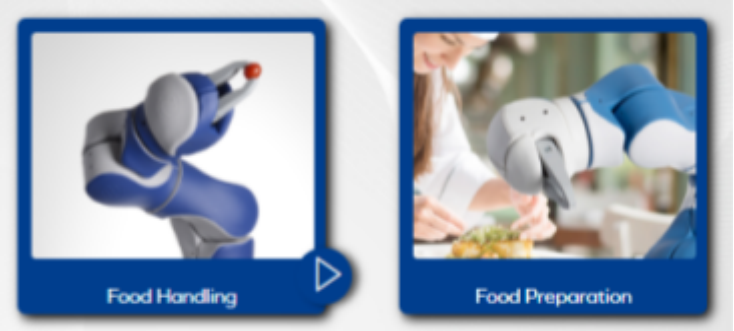


Figure 28: Application Buttons

5. Robot Calibration

Before moving a robotic system, it is always necessary to calibrate each one of the available motors. Before P-Rob is calibrated, it does not know the initial state of its joints, or in other words it doesn't know "which way is up and which way is down". Hence a calibration routine is required to avoid runtime errors and specially to protect the user and the robot from executing unintended movements.

Since P-Rob does not incorporate absolute encoders, the calibration procedure requires for each joint to be calibrated individually. This currently is different for different types of P-Robs. Basically we support the following versions:

- Older versions of P-Rob do **not come with Limit Switches**, which means that the calibration routine drives all motors into their mechanical limits in order to calibrate. This calibration therefore is rather slow.
- Newer versions of P-Rob **include Limit Switches in all joints**, which requires each motor to drive into one of those switches during the calibration routine. This calibration therefore is quite a bit faster.
- Future versions of P-Rob will come with so-called **Half-solute Encoders** (encoder ring in each joint), allowing the motors to move only a few degrees to achieve full calibration. This calibration will only take a few seconds.

On request, a robot can be upgraded with Limit Switches or Half-solute Encoders.

Default Calibration

The standard method of calibrating P-Rob requires the user first to put the robotic arm in a more or less upright position and then execute the default calibration script (in the connection menu choosing **Default Calibration** and pressing **Calibrate**). This procedure calibrates all joints of P-Rob, beginning with the wrist joints and continuing with the elbow joints. For wrist joints, the calibration simply moves these joints until they are properly calibrated, and back again to its calibrated zero position. For elbow joints, the calibration moves all elbow joints with alternating direction into their mechanical stops, and then moves all elbow joints to zero simultaneously. If the end-effector contains an actuator, this actuator will be calibrated last. The resulting positions of the joint angles are the new default zero angles, which are defined as the joint angles that position the robot in a way such that all wrist joints have the same axis of rotation and all elbow joints have parallel axes of rotation (for details about the coordinate systems, refer to the chapter Mathematical Conventions).

To use this calibration method, as for all other applications that require P-Rob to move, please make sure in advance that the robot will be able to move freely and that it avoids touching equipment or itself during the movement.

Example: Default Calibration for P-Rob 2R

- calibrating wrist joints (joints 1, 4 & 6)
- calibrating elbow joints (joints 2, 3 & 5)
- calibrating end-effector actuator (e.g. P-Grip)

Custom Calibration

If you are not able to use the default calibration, e.g. due to lack of space, you are required to write a custom calibration script. For this purpose, connect myP to P-Rob and create an application of type **Calibration Procedure**. Then use the functions defined in the calibration section of the myP Script Functions Manual to calibrate all motors. All calibration scripts appear in the calibration choice drop-down list in the connection window. Navigate to the connection menu and execute the custom calibration procedure there.

A custom calibration is only considered to be successful if it calibrates all of the available motors of the robot and its external devices, and if motors are not moved using standard movement functions before they are calibrated.

6. Mathematical Conventions

Coordinate Systems

For the manipulation of P-Rob in myP, only two coordinate systems are important to the user:

- The **Base Frame** of a P-Rob is defined, such that its z-axis points away from the base plate and the x-axis is parallel to the base plate and points away from the connectors at the back panel of the base. The well-known right-hand rule then defines the direction of the y-axis.
- The **Tool Frame** of a P-Rob differs for each type of end-effector. By default myP supports these three different tool frames:
    - If no P-Grip is mounted, the origin of the tool frame is located in the center of the connection plate between P-Rob and P-Grip. Assuming the robot is standing straight upright, the orientation of the tool frame matches the orientation of the base frame.
    - If a P-Grip is mounted, the origin of the tool frame is located between the fingertips. Assuming the robot is standing straight upright and the gripper is mounted such that the fingers point in the direction of the front LED button at the base of the robot, then the orientation of the tool frame matches the orientation of the base frame.

For an example of these coordinate systems, please refer to the following figures, which show the base and tool coordinate frames for P-Rob 2. For other P-Rob types and for other end-effectors, the same coordinate conventions apply.

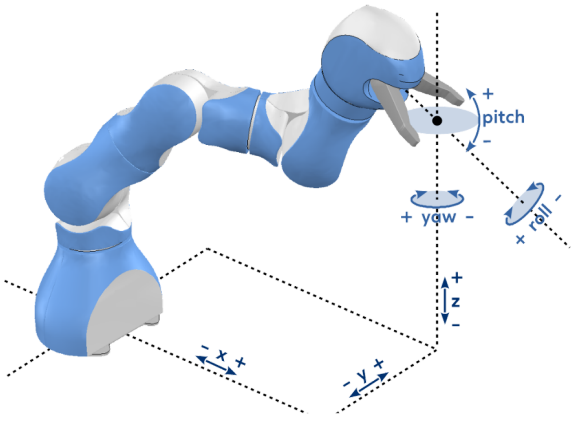


Figure 29: Base Frame for P-Rob 2

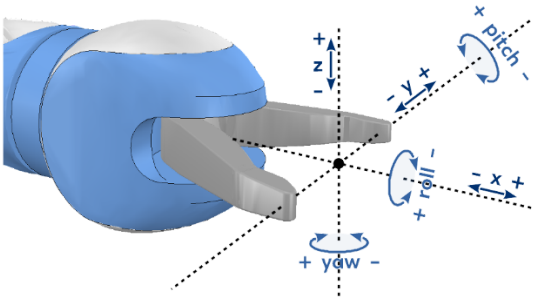


Figure 30: Tool Frame for P-Rob 2

Pose

The pose of a coordinate frame is defined as a six-dimensional array containing both the three-dimensional origin position (given by **x**, **y**, and **z**) as well as the three-dimensional orientation (given by **roll**, **pitch**, and **yaw**) of this frame. Whenever a function in myP uses a pose as an input or output, this pose is defined to be in the base frame of P-Rob.

Orientation Convention

P-Rob 2R has six degrees of freedom. By using three of those degrees to fix the position in space, three degrees remain to be used for the orientation. For the definition of this three-dimensional orientation we use nautical angles, which are commonly used for the orientation of airplanes. Using nautical angles, a three-dimensional rotation is defined as

- 1st a rotation around the negative z-axis in yaw (ψ) direction,
- 2nd a rotation around the negative y-axis in pitch (θ) direction and
- 3rd a rotation around the positive x-axis in roll (ϕ) direction.

Hint: Whenever a function in myP requires you to deal with three dimensional orientations of the end-effector, it helps to imagine a plane being attached to the end-effector, such that the coordinate system of both the end-effector and the plane coincide. This way it is much easier to extract the angles (roll, pitch, and yaw) that lead to this very orientation.

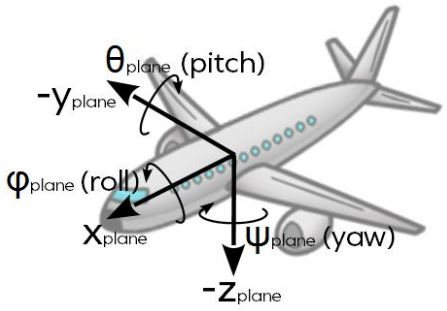


Figure 31: Nautical Coordinate System

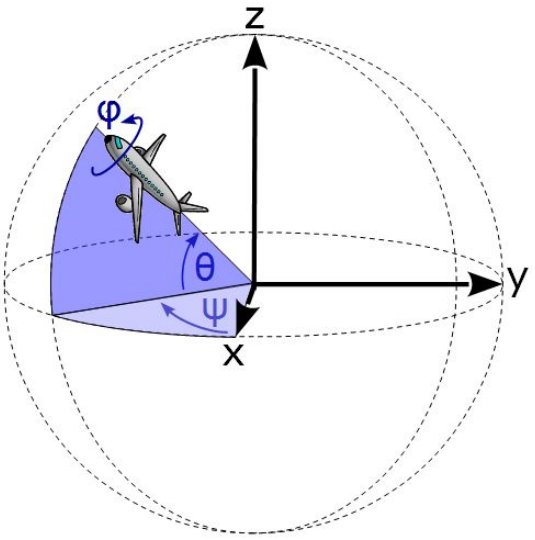


Figure 32: Nautical Order of Rotations

Example: If you use a gripper (e.g. P-Grip) as your end-effector and you define the orientation vector at the TCP such that it points in x-direction, y-direction, or z-direction of the gripper coordinate system, you will not be able to choose the roll, pitch, or yaw angle. Let's say you decide to choose the x-direction (and therefore the inability to select the roll angle of the gripper) and want to pick and place an object lying on the ground, then you can simply move to the position with the orientation [0, 0, -90] to pick the object. In this case, you would not be able to choose from which side the gripper fingers close around the object.

7. Synchronizing Multiple P-Robs

In a setup with multiple P-Robs, it is important to synchronize their workflow to perform synchronous tasks. In this chapter, we give a few examples on how such a synchronization could be established. Note that the methods described here are not exclusive. Better methods will probably exist, and a lot of fine-tuning is needed to adapt the mentioned methods to the needs of a specific application.

TCP Socket

Probably the most common way to communicate between multiple devices on the same network is by using TCP sockets over an active LAN or WLAN connection. This can easily be achieved by using the myP client and server functions. A connection can be created, through which messages can be received and sent. Also the connection can be closed at any time. The connection is automatically closed if the script terminates. If a connection is to be preserved independently from status changes, the background script can be used.

Messages are encoded with the **UTF-8** encoding standard. Furthermore, messages are separated by the **End of Text** character 0x03 (“\x03” in Python).

Example 1

```
# create a client that connects to ip 192.168.80.2 through port 60001
client = create_tcp_client("192.168.80.2", 60001)
# wait until a message was received from the server and print it
print(client.receive())
# send an answer to the server
client.send("i am your client")
# close the connection
client.close()
```

Create a client and wait until a message is received. Upon receiving a message, send an answer to the server and close the client. Note that the script will run until stopped if there is no server that sends a message.

Example 2

```
# create a server that can be reached through port 60002
server = create_tcp_server(60002)
# wait until a message was received from any client and print it
print(server.receive())
# send an answer to all clients
server.send("i am your server")
# close the connection for all clients
server.close()
```

Create a server and wait until a message is received from any client. Upon receiving a message, send an answer to all clients and close the server. Note that the script will run until stopped if there is no client that sends a message.

The connection does not have to be closed after every message, but can be left open until the robots are done with their task. In order to connect two robots, one of them needs to be the server and one of them needs to be the client.

Digital I/Os

If you connect a device to P-Rob that does not support LAN or WLAN, or if you simply prefer not to use web-sockets for security reasons, you may use the integrated digital inputs and outputs of P-Rob for synchronization. This requires you to wire the input and output ports.

Example

With this example, we can synchronize up to 9 P-Robs by interconnecting them using their digital I/O ports. Follow these steps to prepare the robots and run the code:

- Power off all robots before plugging any cables into the digital I/O ports.
- Assign each of your slave robots a number N (1, 2, 3, …)
- For slave robot N, connect the digital output N of the master to the digital input N of the slave and connect the input N of the master to the output N of the slave.
- Connect a power source to the power ports besides the digital I/Os of each robot. You may use a power source with multiple cables leading from the source to each robot.
- Power on all robots and connect them to myP.
- Copy the server script code into a myP application of your master robot. Specify the number of slave robots at the beginning of this code.
- Copy the client script code into a myP application of each of your slave robots. Make sure to specify the slave number previously assigned at the beginning of each script.
- Run all applications (in any order). The application waits for all the scripts to be started and then continues its program flow.

Master Script

```
# give this master the nr. of slaves
NN = ...

# tell slaves that master is ready
slaves = list(range(1, NN+1))
write_digital_outputs([True] * NN, slaves)

# waiting for slaves to be ready too
while False in read_digital_inputs(slaves):
    wait(0.01)

# tell slaves to continue
write_digital_outputs([False] * NN, slaves)

# write your application here...
print("Master says: Hello world!")
```

Slave Script

```
# give this slave its number
N = ...

# wait for master to be ready
while not read_digital_inputs(N):
    wait(0.01)

# tell master that slave is ready
write_digital_outputs(True, N)

# wait for master to send synchronized
start
while read_digital_inputs(N):
    wait(0.01)

# write your application here...
print("Slave", N, "says: Hello world!")
```